

Trabajo Fin de Grado

Uso de técnicas de aprendizaje automático para
la mejora de imágenes
Using Machine Learning techniques for image
enhancement

Autor

Pablo Doñate Navarro

Directores

Dra. D^a Piedad Garrido Picazo

Dr. D. Alfonso Blesa Gascón

Escuela Universitaria Politécnica de Teruel

2022/23



Tabla de Contenidos

1. Introducción y objetivos.....	1
2. Contextualización.....	3
2.1. Funcionamiento de las redes neuronales	3
2.2. Explicación de la técnica de aprendizaje automático seleccionada	6
2.2.1. Redes Generativas Adversarias (GAN).....	6
2.2.2. Redes Generativas Adversarias de Super-Resolución (SRGAN).....	10
3. Estado del arte	17
4. Prototipo de software basado en modelos generativos	19
4.1. Entorno de trabajo	19
4.1.1. Instalación y configuración en Tensorflow	22
4.1.2. Instalación y configuración en PyTorch	23
4.2. Preprocesamiento de los datos	24
4.3. Creación de la red neuronal en Tensorflow.....	26
4.3.1. Red neuronal de partida.....	26
4.3.2. Modificaciones realizadas	30
4.3.3. Problemas encontrados.....	35
4.4. Creación de la red neuronal en PyTorch.....	38
4.4.1. Red neuronal de partida.....	38
4.4.2. Modificaciones realizadas	39
4.4.3. Problemas encontrados.....	43
5. Resultados	44
6. Licencia software y documental	49
7. Conclusiones y trabajo futuro	50
7.1. Trabajo futuro.....	50
8. Bibliografía	51
Anexo I. Conceptos básicos de una red neuronal	I
Anexo II. Retropropagación	IV

Índice de figuras

Figura 1. Arquitectura de una red neuronal	3
Figura 2. Función Sigmoide	4
Figura 3. Arquitectura de una red GAN	6
Figura 4. Entrenamiento discriminador	7
Figura 5. Entrenamiento generador	7
Figura 6. Arquitectura de la red SRGAN	10
Figura 7. Funcionamiento de la capa de convolución	11
Figura 8. Pixel Shuffler.	13
Figura 9. Funciones de activación no-lineales.....	14
Figura 10. Máquina utilizada durante el entrenamiento de la red.	20
Figura 11. Página de PyTorch para instalar su librería en local.	23
Figura 12. Imágenes del dataset seleccionado.....	24
Figura 13. Pseudocódigo del script que permite reescalar las imágenes del dataset.....	25
Figura 14. Pseudocódigo asociado a la preparación del dataset.....	26
Figura 15. Pseudocódigo asociado al preentrenamiento del generador.....	27
Figura 16. Pseudocódigo asociado a la definición del modelo del generador.....	28
Figura 17. Pseudocódigo asociado al entrenamiento de la SRGAN.	30
Figura 18. Ajuste del tamaño de batch en Tensorflow	31
Figura 19. Ajuste del tamaño de recorte en Tensorflow	32
Figura 20. Ajuste del número de filtros del generador en Tensorflow	33
Figura 21. Ajuste del número de bloques residuales del generador en Tensorflow	34
Figura 22. Comparativa del RMSE en el entorno Tensorflow con operaciones en CPU y GPU. ..	35
Figura 23. Resultado de utilizar un número de filtros muy grande.....	36
Figura 24. Comparativa de resultados al utilizar un LR alto y un LR óptimo.	37
Figura 25. Comparativa de resultados al utilizar un LR bajo y un LR óptimo.	37
Figura 26. Pseudocódigo asociado a las funciones de gestión del dataset	38
Figura 27. Ajuste del tamaño de recorte en PyTorch	40
Figura 28. Error por falta de memoria con un tamaño de batch superior a 1.	41
Figura 29. Ajuste del número de filtros en el generador en PyTorch	41
Figura 30. Ajuste del número de bloques residuales en PyTorch.....	42
Figura 31. Comparativa del RMSE en el entorno PyTorch con operaciones en CPU y GPU.	43
Figura 32. Error cometido por el discriminador con las imagenes reales.	44
Figura 33. Error cometido por el discriminador con las imagenes generadas.	45
Figura 34. Pérdida del generador basada en la pérdida perceptual.	45
Figura 35. Error cuadrático medio obtenido en las validaciones del entrenamiento.	46
Figura 36. Logotipo de la licencia BSD.....	49
Figura 37. Arquitectura de una neurona artificial	I
Figura 38. Función de activación Escalón.....	II
Figura 39. Puerta lógica AND.....	II
Figura 40. Puerta lógica OR	II
Figura 41. Puerta AND sobre el plano	III
Figura 42. Puerta XOR.....	III

Índice de tablas

Tabla 1. Resumen de los artículos seleccionados para el estado del arte.....	18
Tabla 2. Tabla resumen del dataset.	24
Tabla 3. Modificaciones realizadas en Tensorflow para el procesamiento monocromático.....	30
Tabla 4. Modificaciones realizadas en PyTorch para el procesamiento monocromático	39
Tabla 5. Imágenes obtenidas en Tensorflow.....	47
Tabla 6. Imágenes obtenidas en PyTorch.	48

Resumen

El propósito de este Trabajo Final de Grado (TFG) consiste en evaluar diferentes técnicas de aprendizaje automático para mejorar imágenes, así como la selección de una de ellas para su posterior evaluación del rendimiento en diferentes entornos de trabajo. Concretamente, se va a centrar en el campo de estudio del procesamiento de imágenes monocromáticas.

En este sentido, se ha obtenido un conjunto de imágenes radiológicas monocromáticas para mejorar su procesamiento mediante el uso de redes generativas adversarias de super-resolución (SRGAN). Estas redes tratan de reconocer y aprender características de las imágenes de forma que al aumentar la resolución de las mismas, no se pierda información.

Para ello, se han realizado dos implementaciones de redes SRGAN para dos librerías de Python denominadas Tensorflow y PyTorch. Para cada una de ellas, se han llevado a cabo diferentes procesos de entrenamiento de la red obteniendo un conjunto de interesantes resultados.

Palabras clave

Super-resolución, Redes Generativas Adversarias, convolución, error cuadrático medio, imágenes monocromáticas.

Abstract

The purpose of this Final Degree Project is to evaluate different machine learning techniques for image enhancement and analyze their performance in various working environments. The goal is to contribute to the current knowledge of machine learning applications in monochrome image processing and serve as a starting point for future research in this field.

In this sense, a set of monochromatic radiological images has been obtained to improve their processing through the use of super-resolution generative adversarial networks (SRGAN). These networks try to recognize and learn features of the images so that when increasing their resolution, no information is lost.

For this purpose, two implementations of SRGAN networks have been carried out for two Python libraries called Tensorflow and PyTorch. For each one of them, different training processes of the network have been carried out obtaining a set of interesting results.

Keywords

Super-resolution, Generative Adversarial Networks, convolution, Root-Mean-Square Error, monochrome images.

1. Introducción y objetivos

Uno de los campos tecnológicos más fascinantes y de rápido desarrollo en la actualidad es la Inteligencia Artificial (IA). Se trata de ^[1] crear algoritmos y sistemas que sean capaces de imitar competencias de la inteligencia humana, como la percepción, el razonamiento ^[2] y la toma de decisiones. Desde la fabricación y la banca hasta la medicina y la agricultura, la IA se utiliza en una amplia gama de sectores industriales.

El campo de la IA conocido como aprendizaje automático (en inglés *Machine Learning*), es uno de los más interesantes. El Machine Learning se refiere al desarrollo de algoritmos que pueden aprender de los datos sin estar programados explícitamente para hacerlo. En lugar de ello, los algoritmos son entrenados con información para encontrar patrones y relaciones en los mismos. Las aplicaciones del aprendizaje automático van desde la detección de fraudes hasta la recomendación de productos, así como la traducción automática.

Las redes neuronales artificiales (en inglés *Artificial Neural Networks*) son un método de aprendizaje automático común y efectivo. Éstas se inspiran en la forma en que funciona el cerebro humano para procesar la información. Se componen de capas de nodos interconectados, donde cada nodo representa una "neurona" sintética que procesa los datos de entrada antes de enviarlos a la siguiente capa. Debido a su capacidad para aprender patrones complejos y conexiones entre datos, las redes neuronales son adecuadas ^[3] para una amplia gama de aplicaciones, incluida la mejora de imágenes, temática en la que se centra este Trabajo Fin de Grado (TFG).

Existen numerosos usos para las técnicas de aprendizaje automático y líneas de trabajo en la mejora de imágenes. En el campo de la medicina ^[4], estas técnicas pueden mejorar las imágenes que se utilizan para la toma de decisiones médicas precisas. En el campo de la vigilancia ^[5], para identificar y reconocer objetos y personas, puede ser crucial mejorar las imágenes con el fin de facilitar la tarea y mejorar el reconocimiento. En la creación de contenido audiovisual ^[6], mejorar la imagen puede enriquecer la experiencia del usuario y hacer que el contenido sea más atractivo visualmente.

Dentro del aprendizaje automático, existen diferentes líneas de trabajo, siendo el deep learning (aprendizaje profundo) el que está tomando un mayor protagonismo ^[7]. Es en esta línea de trabajo donde se ubican las Redes Generativas Adversarias de Super-Resolución (SRGAN), una particular implementación de las Redes Generativas Adversarias (GAN) que a partir de imágenes en baja resolución, generan imágenes de alta resolución.

Es por ello que una vez analizadas una serie de técnicas y líneas en el estado del arte, se opta por las SRGAN para un estudio en profundidad.

Este TFG plantea varios objetivos. Por un lado, está la puesta en marcha del entorno de trabajo, incluyendo la configuración hardware y software de la máquina utilizada, el preprocesamiento de los datos, y la configuración de los diferentes entornos de desarrollo. Por otro lado, está proporcionar un estudio de algunas técnicas de Machine Learning para la mejora de imágenes, así como la selección de una de ellas, evaluando la misma implementación SRGAN en los entornos de trabajo PyTorch y Tensorflow para sacar conclusiones sobre su rendimiento.

El presente documento se estructura en varias secciones. En la sección 2 se contextualiza el trabajo, explicando el funcionamiento de las redes neuronales, así como la tipología seleccionada (SRGAN). En la sección 3 se presenta el estado del arte en el campo de estudio. Por otro lado, en la sección 4 se detalla la propuesta propia, donde se lleva a cabo una implementación de SRGAN para dos entornos de desarrollo: Tensorflow y PyTorch. En esta sección también se describe el dataset utilizado, su preprocesamiento y la creación de la red neuronal en ambos entornos, incluyendo las modificaciones realizadas y los problemas encontrados. En la sección 5 se presentan los resultados obtenidos y se realiza una comparativa entre las dos librerías utilizadas. En la sección 6, se presenta la licencia de software y la licencia documental que se ha escogido para este trabajo. Finalmente, en la sección 7 se presentan las conclusiones y posibles líneas de trabajo futuro.

2. Contextualización

Las redes neuronales son un campo de estudio en constante evolución que ha experimentado un gran avance en las últimas décadas. Estas estructuras computacionales están inspiradas en el funcionamiento del cerebro humano y han demostrado ser herramientas poderosas en diversos campos como el reconocimiento de patrones, el procesamiento de lenguaje natural y la toma de decisiones.

A lo largo de los años, las redes neuronales han experimentado importantes avances y refinamientos, lo que ha permitido su aplicación en una amplia gama de problemas complejos. Estas redes se componen de unidades interconectadas llamadas neuronas artificiales o perceptrones, que trabajan en conjunto para procesar la información y generar resultados.

En esta sección de contextualización, se van a describir los fundamentos de las redes neuronales, su funcionamiento básico, su evolución a lo largo del tiempo, y también se va a explicar las arquitecturas de las redes que van a ser utilizadas y evaluadas en este proyecto. A través de este breve repaso, se pretende sentar las bases necesarias para explorar con mayor profundidad su aplicación en el ámbito de estudio de este TFG.

2.1. Funcionamiento de las redes neuronales

Una red neuronal es un conjunto de neuronas, por lo que si se quiere conocer su funcionamiento, primero se debe conocer el funcionamiento de una neurona (ver Anexo I).

Una vez definidos los conceptos básicos de una red neuronal en el anexo, se va a detallar otro concepto fundamental de las mismas, como es el de “Capa”. Para el tratamiento de las neuronas de una red, las redes se organizan en capas. Una capa es un conjunto de neuronas que no tienen relación entre sí, pero que tienen comportamientos similares. Toda red neuronal tiene que estar formada como mínimo por dos capas: una capa de entrada y una capa de salida, pero se pueden añadir tantas capas intermedias (u ocultas) entre la de entrada y la de salida como se quiera. Como se puede ver a continuación, las salidas de las neuronas de una capa se transmiten como entradas a las neuronas de la capa siguiente, cada una con sus pesos y *bias*:

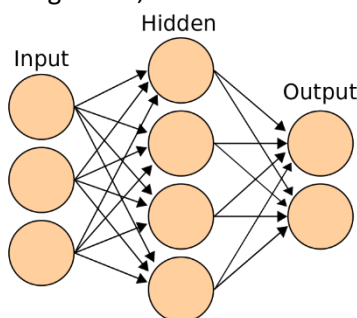


Figura 1. Arquitectura de una red neuronal ^[11]

A la hora de trabajar con una red neuronal, la capa de entrada no se suele contabilizar, por lo que una red neuronal formada por una capa de entrada y una capa de salida se considera “Unicapa”.

En la figura 1 se puede ver fácilmente la idea de paralelismo en las redes neuronales, ya que en una misma capa, cada neurona no necesita de las demás para trabajar, son capaces, por lo tanto, de trabajar simultáneamente.

También, se debe de tener en cuenta lo siguiente: el número de neuronas que debe de tener la capa de entrada, debe de ser igual al número de variables que se tenga en el problema; y en la capa de salida, el número de neuronas debe de ser igual al número de soluciones que tenga el problema. Hay que destacar que cuantas más capas y más neuronas tenga la red, no siempre será mejor el resultado, aunque sí que será mayor el tiempo de entrenamiento y de ejecución, por lo que hay que buscar un punto medio, donde se obtengan buenos resultados en un tiempo aceptable, teniendo en cuenta además los recursos de los que se dispone.

La evolución de las redes neuronales ^[12] ha sido un proceso fascinante que ha llevado a grandes avances en el campo de la inteligencia artificial. A lo largo de las décadas, estas redes han experimentado transformaciones significativas en su diseño, arquitectura y capacidades, impulsadas por descubrimientos científicos, avances tecnológicos y desafíos cada vez más complejos. Para ello, se considera oportuno hacer en breve repaso sobre la misma

- En el año 1958, el científico Frank Rosenblatt, creó el **perceptrón**, la unidad desde donde nacería y se potenciarían las redes neuronales artificiales (ANN). Un perceptrón toma varias entradas binarias y produce una sola salida binaria.
- Una vez apareció el perceptrón, siete años después (año 1965) aparecerían redes neuronales formadas por perceptrones, denominadas **perceptrones multicapa (MLP)**. Surge el concepto de capas de entrada, ocultas y de salida. En este caso, se siguen manteniendo valores de entrada y de salida binarios. Los valores de los pesos eran asignados manualmente por cada desarrollador.
- En la década de los 80, aparecen las **redes neuronales feedforward (FNN)**. Estas redes constan de una capa de entrada, una capa o capas ocultas y una capa de salida. Aunque estas ANN se confunden con frecuencia con las redes MLP, debe tenerse en cuenta que mientras que en los perceptrones multicapa, las funciones de activación pueden ser únicamente no lineales, las redes FNN pueden tener diferentes tipos de funciones de activación, incluyendo funciones lineales y no lineales.

Un ejemplo de función no lineal es la **función sigmoide** $f(x) = \frac{1}{1 + e^{-x}}$, que se utiliza comúnmente como función de activación en redes neuronales. Esta función toma un valor de entrada y lo comprime en un rango comprendido entre 0 y 1, lo que la convierte en una función útil para modelar probabilidades y realizar clasificaciones binarias.

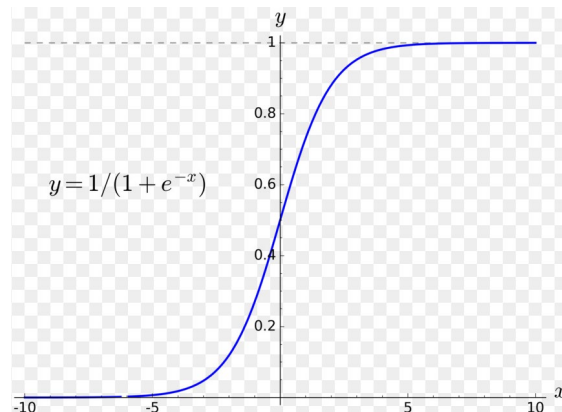


Figura 2. Función Sigmoide ^[13]

- Además, en 1986 aparece el algoritmo de **retropropagación** o *backpropagation* (ver Anexo II), el cual permite entrenar redes neuronales de varias capas de forma supervisada. Este algoritmo calcula el error obtenido en la salida y lo propaga hacia las capas anteriores para hacer ajustes pequeños en cada iteración, minimizando el coste y permitiendo que la red aprenda. De esta manera, la red puede clasificar correctamente las entradas y mejorar su precisión en tareas de clasificación u otras tareas.
- En el año 1989, surgen las **redes neuronales convolucionales (CNN)**. Estas redes son similares a las redes feedforward, pero normalmente se utilizan para el reconocimiento de imágenes, reconocimiento de patrones y/o visión artificial. Utilizan los principios del álgebra lineal, especialmente la multiplicación de matrices, para detectar patrones en las imágenes.
- Fue en el año 2006 cuando se alcanzó el **aprendizaje profundo (deep learning)**. Hasta ahora, las redes neuronales inicializaban de forma aleatoria los pesos, y aquellas que estaban formadas por múltiples capas, podían caer en mínimos locales sin lograr optimizar los pesos. Fue en este año cuando surgió la idea del preentrenamiento de las capas de manera no supervisada. De esta forma, se conseguía tener redes neuronales con múltiples capas ocultas que obtenían buenos resultados.
- Finalmente, en el año 2014 aparecen las **redes neuronales generativas adversarias (GAN)**, que es un modelo formado por dos redes neuronales que están compitiendo. Una de ellas denominada Generador, que con “datos basura” genera muestras, y la otra red, denominada Discriminador, que a partir de las muestras del generador y las muestras del conjunto de entrenamiento (real), debe diferenciar entre las dos entradas cuál es la real y cuál es la generada. De esta forma, el generador aprende a producir muestras más realistas y el discriminador aprende a distinguir entre datos reales y muestras artificiales.

2.2. Explicación de la técnica de aprendizaje automático seleccionada

En este proyecto se ha llevado a cabo un estudio de diferentes técnicas de aprendizaje automático que permiten la mejora de imágenes. La técnica que se ha seleccionado es la Red Generativa Adversaria de Super-Resolución (SRGAN). Sin embargo, para poder introducir este tipo de red neuronal, antes se va a comenzar definiendo las redes generativas adversarias (GAN), ya que las SRGAN son un tipo concreto de éstas.

2.2.1. Redes Generativas Adversarias (GAN)

Hasta ahora, las redes neuronales que se han descrito son muy simples. Sin embargo, el proyecto se centra en redes que van más allá de las características de las unicapa o multicapa.

Las redes GAN ^[14] están formadas por una arquitectura de dos redes neuronales, conocidas como red generadora y red discriminadora. La red generadora tiene como objetivo crear nuevas instancias del mismo dominio que el conjunto de datos original, y la red discriminadora es la responsable de reconocer si los datos de entrada son reales (pertenecen al conjunto de datos de entrada) o son ficticios (generados artificialmente). Ambas redes se entrenan juntas, de tal manera que el generador maximiza las posibilidades de no ser detectado por el discriminador, y el discriminador genera cada vez métodos de detección más sofisticados.

Tanto el generador como el discriminador pueden ser de diferentes tipos de redes neuronales, dependiendo de la tarea específica para la que se esté utilizando la GAN. Por ejemplo, si la tarea es generar imágenes, es común utilizar redes neuronales convolucionales tanto para el generador como para el discriminador. Las redes convolucionales son especialmente efectivas para procesar datos visuales debido a su capacidad para detectar patrones locales y jerarquías de características en las imágenes. Sin embargo, hay una característica común que siempre tienen las redes GAN, y es que utilizan el algoritmo de retropropagación para actualizar los pesos y las *bias* tanto del generador como del discriminador en función del error calculado.

La arquitectura básica de esta red ^[15], es la que puede verse en la siguiente figura, donde se aprecian tanto la red generadora como la red discriminadora:

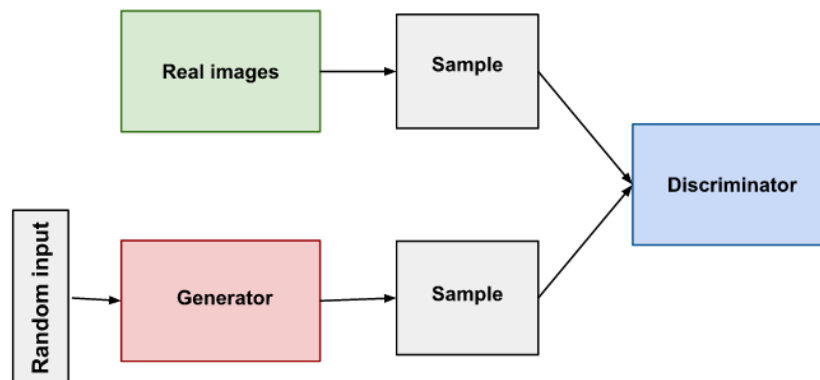


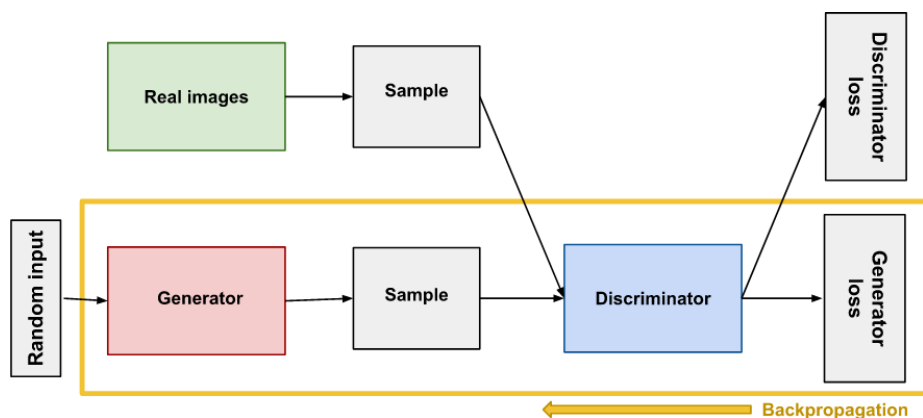
Figura 3. Arquitectura de una red GAN ^[15]

A partir de un conjunto de imágenes (*Real Images*), se obtiene una imagen denominada muestra (en inglés, *Sample*). Por otro lado, el generador recibe como entrada “datos basura” también conocidos como “*random input*”. Con ellos, el generador produce muestras artificiales que se asemejen a los datos reales. De esta forma, el discriminador recibe tanto muestras del conjunto de datos reales, como muestras producidas por el generador. El objetivo del

discriminador es diferenciar entre las muestras reales y las muestras artificiales producidas por el generador.

Durante el entrenamiento del discriminador, el generador no se entrena. Sus pesos permanecen constantes mientras produce ejemplos, los cuales se usan para entrenar al discriminador. El discriminador asigna una probabilidad a cada muestra que indica si cree que es real o artificial. Luego, se calcula el error entre la salida del discriminador y la salida deseada (1 para las muestras reales y 0 para las muestras artificiales) y se utiliza el algoritmo de retropropagación para actualizar los pesos y las *bias* del discriminador en función de este error:

Figura 4. Entrenamiento discriminador [15]



El generador y discriminador tienen procesos de entrenamiento diferentes. A la hora de trabajar con ambas redes, la red GAN debe ser entrenada como un todo, por lo que el proceso de entrenamiento de la GAN consistirá en los siguientes pasos:

1. Se inicia una época de entrenamiento.
2. Se selecciona una muestra del conjunto de datos reales.
3. Se genera una muestra artificial utilizando el generador.
4. Se entrena el discriminador en la muestra combinada de muestras reales y artificiales.
5. Se genera otra muestra artificial utilizando el generador.
6. Se entrena el generador utilizando las salidas del discriminador como señales de error.
7. Se repiten los pasos 2-6 para completar un ciclo de entrenamiento hasta que se haya utilizado todo el conjunto de datos reales.
8. Se repiten los pasos 1-7 para completar múltiples épocas de entrenamiento.

Cabe indicar que un ciclo de entrenamiento es una iteración completa del proceso de entrenamiento en la que se entrena tanto el generador como el discriminador con una misma muestra del conjunto de datos. Por otro lado, una época de entrenamiento consiste en completar múltiples ciclos de entrenamiento utilizando todo el conjunto de datos.

Por último, quedaría hablar de las **funciones de pérdida**, que se han mencionado anteriormente y que deben usarse para reflejar cómo son de parecidos los datos producidos por el generador y los datos reales.

Existen múltiples funciones de pérdida que se pueden utilizar, aunque la elección de éstas se verá afectada por el tipo de datos con los que se trabaja. En el caso de utilizar imágenes, las funciones de pérdida serán aquellas que comparen la pérdida que existe en las imágenes generadas con respecto a las imágenes originales.

En una GAN, se puede tener una única función de pérdida común tanto para el generador como para el discriminador, y también se puede tener una función de pérdida para el generador y otra diferente para el discriminador.

Una de las funciones que es muy típica de utilizar en las redes GAN, tanto en el generador como en el discriminador, es la **entropía cruzada binaria** (en inglés *Binary Cross Entropy*), que permite ^[17] medir la diferencia entre la distribución de probabilidad de la red generadora y la distribución de probabilidad de los datos reales. Si esta diferencia es baja, entonces se podrá decir que el generador habrá aprendido a generar datos que se asemejen a los reales. En caso contrario, la pérdida será alta y el generador necesitará ajustar sus parámetros para producir datos más realistas.

La ecuación para calcular la entropía cruzada binaria es la siguiente:

$$H_p(q) = -\frac{1}{N} \sum_{i=1}^N y_i \log(p(y_i)) + (1 - y_i) \log(1 - p(y_i))$$

En el caso de las redes GAN, la entropía cruzada binaria toma como entrada las predicciones del discriminador $p(y_i)$ y las etiquetas verdaderas de las muestras y_i . En el caso de una GAN, las etiquetas verdaderas son 1 para las muestras reales y 0 para las muestras generadas por el generador.

En el contexto de la **teoría de la información** ^[18] de Shannon y Weaver, esta fórmula se puede interpretar como el número promedio de bits necesarios para identificar un evento de un conjunto de posibilidades, si un esquema de codificación está basado en la distribución de probabilidad dada por $p(y_i)$, en lugar de en la verdadera distribución y_i . La primera parte de la fórmula, $y_i \log(p(y_i))$, representa el número promedio de bits necesarios para identificar un evento positivo (etiqueta de clase 1), mientras que la segunda parte, $(1 - y_i) \log(1 - p(y_i))$, representa el número promedio de bits necesarios para identificar un evento negativo (etiqueta de clase 0).

2.2.2. Redes Generativas Adversarias de Super-Resolución (SRGAN)

Una vez explicados estos conceptos: la arquitectura y funcionamiento de las redes GAN, se puede pasar a explicar la implementación conocida como SRGAN.

Las redes generativas adversarias de superresolución, denominadas SRGAN (Super-Resolution Generative Adversarial Network) son ^[19] un tipo específico de red GAN que permite trabajar la resolución de imágenes.

La arquitectura completa de una SRGAN es la que se muestra a continuación, y que va a ser explicada:

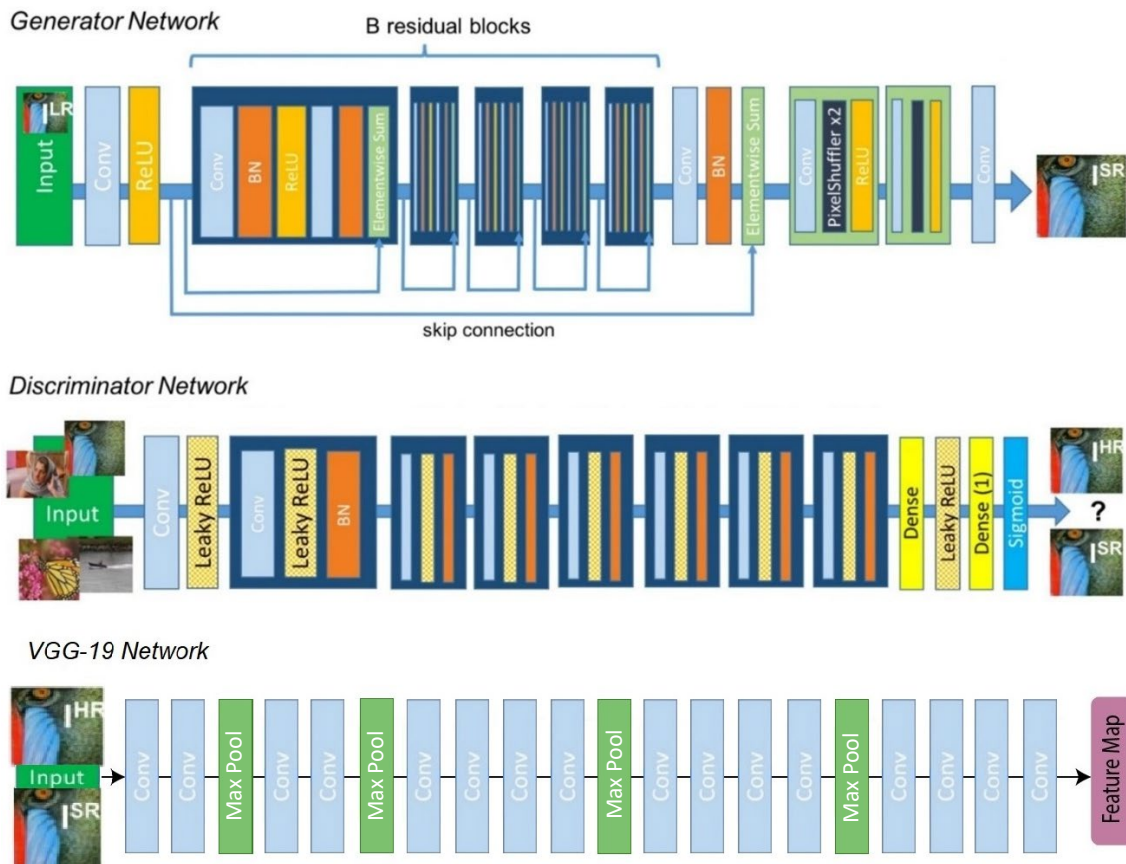


Figura 6. Arquitectura de la red SRGAN ^[19]

La red está formada por un **generador**, un **discriminador**, y una red extractora de características denominada **VGG-19**. En este caso, la red generadora recibe como entrada una imagen en baja resolución (LR) y genera una imagen en super-resolución (SR), mientras que la red discriminadora, recibe imágenes reales y artificiales, y para cada una de ellas tiene que determinar si es una imagen real (HR) o una imagen en super-resolución. Mientras tanto, la red VGG-19 se utiliza para calcular la pérdida perceptual entre las imágenes generadas por la red generadora y las imágenes originales de alta resolución. La pérdida perceptual mide la diferencia entre las características de alto nivel extraídas por la red VGG-19 de las imágenes generadas y las imágenes reales.

La red generadora se trata de una red neuronal convolucional profunda que utiliza capas de convolución para extraer características de las imágenes de baja resolución y luego utiliza capas de submuestreo para aumentar la resolución de las imágenes. Por otro lado, la red discriminadora también es una red neuronal convolucional, y en este caso, utiliza capas de convolución para extraer características de las imágenes y luego utiliza capas completamente conectadas para hacer la clasificación entre imágenes reales y generadas. Finalmente, la red VGG-19 es una red neuronal convolucional pre-entrenada que utiliza capas convolucionales para extraer un conjunto de características de la imagen de entrada.

A continuación, se van a detallar cada una de las partes de la arquitectura: generador, discriminador y VGG-19.

Arquitectura del generador

El generador comienza por una capa convolucional junto con una función de activación ReLU (Rectified Lineal Unit). Esta capa convolucional sirve para extraer características de la imagen. Esto se consigue gracias a un filtro o **Kernel**, el cual se va desplazando por toda la imagen detectando bordes o formas específicas.

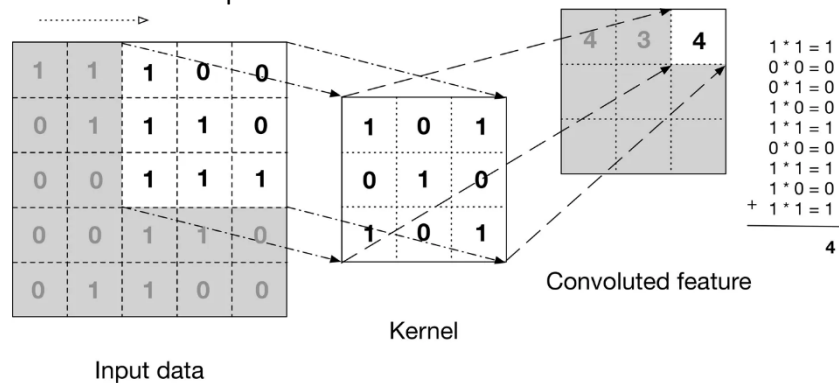


Figura 7. Funcionamiento de la capa de convolución ^[21]

Tal y como aparece en la figura 7, dada una imagen (input data), el Kernel se va desplazando por ella de izquierda a derecha y de arriba a abajo, y para cada posición, se realiza un producto escalar entre los valores de la imagen y los valores del Kernel, generando un valor de salida que se ubica en la posición correspondiente de la imagen de características (*convoluted feature*). El Kernel, realmente es la matriz de pesos, los cuales se van ajustando conforme se entrena la red generadora, de forma que la red ajusta los valores del Kernel para que la salida de la convolución sea lo más cercana posible a la salida deseada. Esta convolución, se combina con una función de activación denominada ReLU, la cual mapea los valores negativos de entrada a cero, y los valores positivos los mantiene iguales.

Algo a destacar de las capas convolucionales, es que se les debe indicar el **número de filtros**, uno de los hiperparámetros a ajustar en la arquitectura SRGAN. Los **hiperparámetros** son configuraciones o valores numéricos que se establecen previamente al entrenamiento de la red y afectan el proceso de aprendizaje. Aunque no se aprenden directamente del conjunto de datos, desempeñan un papel fundamental en el rendimiento y los resultados del modelo.

Los **filtros** representan un conjunto de patrones o características específicas que la capa convolucional intenta detectar en los datos de entrada. El número de filtros determina la cantidad de características distintas que la capa convolucional puede aprender y, por lo tanto, tiene un impacto directo en la capacidad del modelo para extraer características importantes y realizar representaciones de mayor nivel. Un ejemplo sería el siguiente, donde si se tiene una imagen con resolución (64,64) con 1 canal (imagen monocromática), y el número de filtros indicado es 64, cada uno de estos filtros producirá una imagen de salida de tamaño (64,64), que apiladas verticalmente producirán una matriz de salida de tamaño (64,64) con 64 canales o filtros (64,64,64).

Por otro lado, en las capas convolucionales está el hiperparámetro asociado al **tamaño del Kernel**. Este Kernel tiene dimensiones definidas, como 3x3 o 5x5, que determinan el área local que se considera en cada operación de convolución. Cuanto mayor sea el tamaño del Kernel, mayor será el contexto espacial que se considera al extraer características.

El tamaño del mismo influye directamente en la capacidad del modelo para detectar patrones a diferentes escalas y niveles de detalle. Un tamaño de Kernel más pequeño, como 3x3, tiende a capturar características locales y detalles finos, mientras que un tamaño de Kernel más grande, como 5x5 o 7x7, tiene una vista más amplia y puede capturar características de mayor escala.

Una vez se ha terminado con la primera capa de la red, se pasa a los **bloques residuales**. Estos bloques residuales están compuestos de varias capas convolucionales, cada una seguida de una capa de normalización por lotes y una función de activación ReLU. La entrada a un bloque residual se conecta directamente con la salida del mismo, de forma que se combinan las características extraídas anteriormente, junto con las nuevas características extraídas. Normalmente, estos bloques residuales se utilizan para detectar características más concretas de la imagen (el tamaño del Kernel suele ser menor que el de la primera capa convolucional). El **número de bloques residuales** es otro hiperparámetro fundamental a ajustar en la red generadora. Hay que añadir que el número de bloques residuales afecta directamente a la capacidad de la red para generar imágenes de superresolución. Si la red tiene un número insuficiente de bloques residuales, la imagen generada puede tener detalles insuficientes o aparecer borrosa. Por otro lado, si la red tiene un número excesivo de bloques residuales, esto puede llevar a un mayor tiempo de entrenamiento.

La **normalización por lotes** que se aplica después de cada capa de convolución, se utiliza para normalizar todos los valores (activaciones) de cada canal o filtro, lo que ayuda a mantener todos los valores en una escala similar.

Una vez se ha terminado con los bloques residuales, entran en juego las **capas de aumento de muestreo** (en inglés, *upsample layers*). Estos bloques serán los encargados de aumentar la resolución de la imagen mediante una técnica denominada "*Pixel Shuffler x2*", en la cual se divide cada pixel de la imagen de entrada en bloques de tamaño 2x2, y luego reorganiza estos bloques para crear una imagen de salida con el doble de la resolución en ambas direcciones. Específicamente, cada bloque de tamaño 2x2 se reorganiza en un bloque de tamaño 1x1, pero con 4 veces el número de canales. Este proceso se realiza mediante una operación de reorganización de píxeles que se conoce como "shuffle".

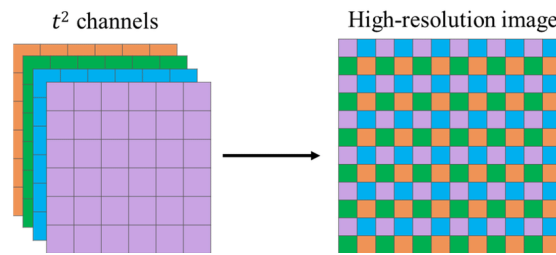


Figura 8. Pixel Shuffler. ^[22]

De esta forma, el número de bloques upsample vendrá dado por el factor de multiplicación de resolución deseado. En el caso de un factor de multiplicación de 2, como cada bloque upsample duplica la resolución de una imagen, solo se necesitará un bloque. En el caso de un factor de multiplicación de 4, se necesitarán 2 bloques de upsample, y así sucesivamente.

Finalmente, la **red generadora** tiene una última capa convolucional encargada de combinar las características extraídas por las capas convolucionales anteriores, y generar una imagen con n canales, siendo n el número de filtros que se aplica a la misma. En el caso de trabajar con imágenes monocromáticas, n será 1, y en caso contrario, el número de canales será 3 (RGB, tres canales de color: rojo, verde y azul). De esta forma, quedaría definida la red generadora.

Arquitectura del discriminador

La **red discriminadora** se encarga de determinar si la imagen de entrada es real o producida por la red generadora. Para hacer esto, la red discriminadora utiliza **bloques discriminatorios**, que son una serie de capas convolucionales que se aplican a la imagen de entrada.

Cada bloque discriminatorio consiste en una capa convolucional seguida de una capa de normalización por lotes y la **función de activación LeakyReLU**.

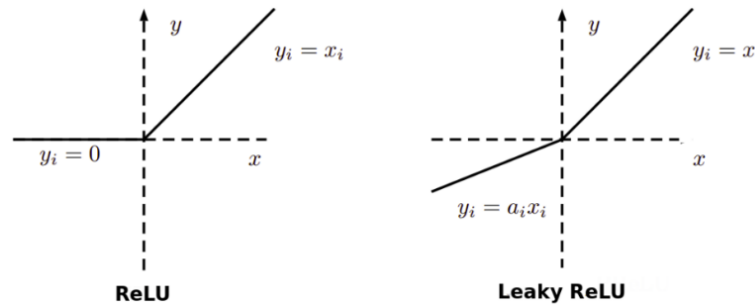


Figura 9. Funciones de activación no-lineales. [20]

La función de activación LeakyReLU es similar a ReLU en el sentido de que devuelve el valor de entrada para valores positivos. Sin embargo, a diferencia de ReLU, LeakyReLU no apaga completamente los valores negativos. En su lugar, LeakyReLU los ajusta por un factor pequeño (por ejemplo, 0.2 o 0.01), lo que evita que los valores negativos tengan un impacto excesivamente negativo en la salida de la función.

Este enfoque de escalar los valores negativos en lugar de apagarlos por completo tiene beneficios significativos durante el entrenamiento de la red. Permite que la información en los valores negativos se transmita a través de la red, lo que contribuye a una mejor propagación de la información relevante y una adaptación más eficaz de los pesos de la red.

Los bloques discriminatorios se utilizan para extraer características de la imagen de entrada y para que la red aprenda a distinguir las características comunes de las imágenes reales. Cabe destacar que entre los bloques discriminatorios, se van alternando el número de strides entre 1 y 2. Los **strides** son, junto con el número de filtros y el tamaño del Kernel los hiperparámetros de la red discriminadora. En este caso, los strides son el número de píxeles que se mueve cada iteración el Kernel. Si se usa un stride igual a 1, el Kernel irá recorriendo, de uno en uno, todos los píxeles. Sin embargo, con un valor de stride 2, el filtro se mueve dos píxeles cada vez, lo que provoca que la imagen de salida sea la mitad de ancha y alta. De esta forma, se consigue reducir progresivamente el tamaño de la imagen mientras que el número de filtros aumenta. Esto se hace para capturar características de mayor nivel y más abstractas en la imagen.

Al finalizar con los bloques discriminatorios, hay una **capa densa** que tiene como función tomar las características de alto nivel aprendidas en las capas anteriores y producir una única salida que indique si la imagen es real o generada. Esta capa tiene una gran cantidad de neuronas, lo que permite que la red discrimine con una alta precisión. La función de activación de esta capa nuevamente es la función LeakyReLU. Las neuronas procesan la información de las características extraídas para producir una salida que se alimenta a una capa sigmoide, que proporciona la probabilidad de que la imagen de entrada sea real o generada.

Arquitectura de la red VGG-19

La red VGG-19 es una red convolucional pre-entrenada que permite extraer características ^[19] de la imagen de entrada, y está formada por bloques convolucionales. Al tratarse de una red pre-entrenada, no va a ser posible ajustar los hiperparámetros de la misma. Cabe resaltar que es utilizada tanto por PyTorch como por Tensorflow.

Cada **bloque convolucional** está compuesto por un número progresivo de capas convolucionales con filtros de tamaño 3x3 y una capa final de max-pooling . Estos bloques permiten extraer características de las imágenes de entrada en diferentes niveles de abstracción. Las capas convolucionales, al igual que ocurre en la red generadora, aplican filtros a la imagen de entrada para detectar patrones específicos, como bordes, esquinas y objetos. A medida que la información fluye a través de las capas convolucionales, las características extraídas se vuelven más abstractas y representativas del contenido semántico de la imagen.

Las capas de **max-pooling** se utilizan para reducir el tamaño de la imagen de características o mapa de activación que se van obteniendo a medida que la información fluye a través de las capas convolucionales. Estas capas se intercalan entre los bloques convolucionales y reducen a la mitad el tamaño de los mapas de activación. El objetivo del max-pooling es reducir la cantidad de datos entre una capa y otra, facilitando así el procesamiento de las imágenes y el entrenamiento de la red, pero a la vez preservando la información más relevante.

La función de pérdida utilizada en las SRGAN se basa en la diferencia perceptual entre las imágenes generadas y las imágenes reales. Aquí es donde la red VGG-19 entra en juego. En lugar de diseñar una función de pérdida completamente nueva, las SRGAN utilizan una función de pérdida perceptual que se basa en la diferencia de características extraídas de las imágenes utilizando la red VGG-19. Esto implica utilizar la red pre-entrenada como un extractor de características y calcular la diferencia entre las características de las imágenes generadas y las imágenes originales de alta resolución.

Por último, se van a indicar las diferentes **funciones de pérdida** utilizadas en las redes que la arquitectura SRGAN, ya que, como se ha indicado, en la retropropagación, donde se actualizan los pesos de las redes, se debe de obtener una función de pérdida para cada una de las redes neuronales.

En el caso del generador, la definición de la función de pérdida de contenido es crítica para el rendimiento del generador. Se denomina **pérdida perceptual**, ^[19] y evalúa cada solución en función de características perceptualmente relevantes. Está formulada como la suma ponderada de una pérdida de contenido y un componente de pérdida adversarial:

$$l^{SR} = l_{VGG}^{SR} + (10^{-3} \cdot l_{Gen}^{SR})$$

Donde l^{SR} es la pérdida perceptual, l_{VGG}^{SR} es la pérdida de contenido, y l_{Gen}^{SR} es la pérdida adversarial.

La **pérdida de contenido**, calculada mediante el error cuadrático medio MSE, es uno de los objetivos de optimización más utilizados para la superresolución de imágenes en que se basan muchos enfoques de vanguardia ^{[19] [21]}. Sin embargo, aunque se logra un alto RMSE, las soluciones de los problemas de optimización del MSE a menudo carecen de contenido de alta frecuencia, lo que resulta en soluciones perceptualmente insatisfactorias con texturas demasiado suaves. En lugar de utilizar una función basada en pérdidas píxel a píxel, se va a utilizar una función de pérdida que se acerca más a la similitud perceptual. Es aquí donde entra en juego la red VGG-19.

$$l_{VGG}^{SR} = \frac{1}{W_{i,j}H_{i,j}} \sum_{x=1}^{W_{i,j}} \sum_{y=1}^{H_{i,j}} (\varphi_{i,j}(I^{HR})_{x,y} - \varphi_{i,j}(G_{\theta_G}(I^{LR}))_{x,y})^2$$

Con $\varphi_{i,j}$ se indica el mapa de características obtenido por la red VGG-19. Las representaciones de características de una imagen reconstruida $G_{\theta_G}(I^{LR})$ y la imagen de referencia I^{HR} . Aquí, $W_{i,j}$ y $H_{i,j}$, describen las dimensiones de los respectivos mapas de características dentro de la red.

Por otro lado, la **pérdida adversarial** provoca que el generador trate de generar imágenes más realistas que sean capaces de engañar a la red discriminadora. Ésta es la siguiente:

$$l_{Gen}^{SR} = \log(D_{\theta_G}(G_{\theta_G}(I^{LR})))$$

Donde $D_{\theta_G}(G_{\theta_G}(I^{LR}))$ es la probabilidad de que la imagen generada sea una imagen real.

Para la función de pérdida del discriminador, se calcula la pérdida del mismo utilizando la función de entropía cruzada binaria (en inglés *Binary Cross Entropy*). En este caso, se calculan dos pérdidas de entropía cruzada binaria, una para las imágenes originales y otra para las imágenes generadas por el modelo, siendo el resultado la suma de ambas, es decir, el error que tiene prediciendo la imagen real más el error que tiene prediciendo la imagen generada.

$$\begin{aligned} l_{Disc} &= l_{Disc}^{HR} + l_{Disc}^{LR} \\ l_{Disc}^{HR} &= \log(D_{\theta_D}(I^{HR})) \\ l_{Disc}^{LR} &= \log(D_{\theta_D}(G_{\theta_G}(I^{LR}))) \end{aligned}$$

Donde $D_{\theta_D}(I^{HR})$ es la probabilidad que determina que la imagen original es realmente real, y $D_{\theta_D}(G_{\theta_G}(I^{LR}))$ es la probabilidad que determina que la imagen producida por el generador es real.

3. Estado del arte

En esta sección, se va a realizar una búsqueda sobre trabajos académicos similares o del mismo campo de estudio que este TFG en los principales repositorios de artículos de investigación: IEEExplorer, Elsevier, ArXiv, Springer y Google Scholar. Los parámetros de búsqueda que se han seleccionado son: “SRGAN”, “superresolución”, “imágenes médicas”, “procesamiento de imágenes médicas”, “redes neuronales convolucionales” y “aprendizaje profundo”.

Para llevar a cabo un análisis de las redes SRGAN en la superresolución de imágenes, es necesario tener una comprensión profunda de su funcionamiento. Por esta razón, se han seleccionado trabajos que proporcionan información lo más detallada y actualizada posible sobre las redes SRGAN, así como otras técnicas de aprendizaje profundo que permitan la mejora de imágenes. Estos trabajos incluyen estudios recientes, artículos de investigación y publicaciones de expertos, lo que respalda la fiabilidad y relevancia de la información seleccionada.

El primer artículo, *“Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network”* ^[19], escrito por Christian Ledig, Lucas Theis, Ferenc Huszar y Jose Caballero, presenta una SRGAN, cuyo objetivo es recuperar los detalles más finos de la textura cuando se realiza la superresolución de imágenes con grandes factores de ampliación. Una prueba exhaustiva de puntuación media de opinión MOS (en inglés, *Mean Opinion Score*) muestra ganancias, enormemente significativas, en la calidad de las imágenes generadas utilizando SRGAN.

El artículo *“Deep Learning for Image Enhancement and Correction in Magnetic Resonance Imaging-State-of-the-Art and Challenges”* ^[22], escrito por Zhaolin Chen, Kamlesh Pawar, Mevan Ekanayake, Cameron Pain, Shenjun Zhong y Gary F. Egan, realiza un estudio de los métodos basados en el aprendizaje profundo para el postprocesamiento de imágenes de resonancia magnética con el objetivo de mejorar su calidad. El postprocesamiento de este tipo de imágenes reconstruidas, a menudo se automatiza y juega un papel crítico en la calidad final de la imagen para informes e interpretación clínica.

En el Trabajo Final de Grado (TFG) con título *“Aplicación de redes neuronales convolucionales profundas al diagnóstico asistido de la enfermedad de Alzheimer”* ^[23], Ángel de la Vega Jiménez, propone el desarrollo de una red neuronal convolucional para clasificar imágenes cerebrales de emisión de positrones y resonancia magnética en tres clases, según el grado de evolución de la enfermedad de Alzheimer. El autor estudia la profundidad adecuada de las redes para este problema, la importancia del preprocesado de las imágenes médicas, el uso de técnicas de transferencia de aprendizaje y aumento de datos para reducir los efectos del problema que supone tener pocos datos, y el uso simultáneo de múltiples modalidades de imagen médica. Su modelo final, que hace uso de la transferencia de aprendizaje con datos de COVID-19, consigue una exactitud del 68%. Concluye que el aumento de la profundidad de las redes ayuda en este problema y que el preprocesado de las imágenes y el uso de técnicas de aumento de datos y redes preentrenadas con imágenes de otras enfermedades pueden aportar mejoras notables.

Por otro lado, se encuentra el artículo, *“Underwater Image Enhancement using Convolutional Neuronal Network”* ^[24], escrito por Anushka Yadav, Mayank Upadhyay, Ghanapriya Singh, donde se propone un método para mejorar las imágenes realizadas bajo el agua utilizando el principio

de la ecualización del histograma. Dado que las imágenes submarinas tienen un color dominante fuerte global, su colorido y contraste a menudo se degradan. Antes de aplicar la técnica de ecualización del histograma en la imagen, la imagen se convierte de color a escala de grises. La ecualización del histograma es una técnica para ajustar las intensidades de la imagen mejorando el contraste. Los colores de la imagen se retienen utilizando un modelo de red neuronal convolucional entrenado con un dataset de imágenes submarinas para obtener mejores resultados.

Finalizando con en el artículo “*Convolutional neural networks for medical image analysis: State-of-the-art, comparisons, improvement and perspectives*”^[25], los autores Hang Yu, Laurance T. Yang, Qingchen Zhang, David Armstrong y M. Jamal Deen, realizan un estudio sobre las redes neuronales convolucionales en el análisis de imágenes médicas. Los autores revisan las redes más utilizadas en el procesamiento de imágenes médicas, como por ejemplo, AlexNet, GoogleNet, ResNet, R-CNN y FCNN en tareas de diagnóstico médico como: clasificación de imágenes, segmentación, detección, registro y recuperación de imágenes para cerebro, seno y abdomen. Finalmente, concluyen con una serie de retos y líneas futuras de investigación, resultado del trabajo realizado.

A continuación, se presenta una tabla resumen con los documentos seleccionados para el estado del arte:

Artículo	Fuente	País	Año
<i>Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network</i>	<i>IEEE Conference on Computer Vision and Pattern Recognition (CVPR)</i>	Estados Unidos	2016
<i>Deep learning for image enhancement and Correction in Magnetic Resonance Imaging- State-of-the-art and Challenges</i>	<i>Journal of Digital Imaging</i> (Publicado por Springer)	Australia	2022
Aplicación de Redes neuronales convolucionales profundas Al Diagnóstico asistido de la enfermedad de Alzheimer	TFG de la Universidad de Granada	España	2022
<i>Underwater Image Enhancement Using Convolutional neural Network</i>	<i>Electronic Journals</i> (Publico por MDPI)	China	2021
<i>Convolutional neural networks for medical image analysis: State-of-the-art, comparisons, improvement and perspectives</i>	<i>Neurocomputing</i> (Publicado por Elsevier)	China, Canada	2020

Tabla 1. Resumen de los artículos seleccionados para el estado del arte.

Con ello, se puede concluir que existen un conjunto de técnicas diferentes que permiten la mejora de imágenes en diferentes ámbitos, como son la mejora de calidad o la superresolución. Sin embargo, no se han encontrado trabajos donde se realiza una comparativa de estas técnicas en diferentes entornos de trabajo, ni se detalla la configuración de los mismos.

4. Prototipo de software basado en modelos generativos

La propuesta que se presenta en este TFG se fundamenta no solo en la creación de una red neuronal que dada una imagen en baja resolución, genere la misma imagen en superresolución, es decir, una imagen que mantiene la información de la imagen original, pero en una resolución mayor, sino también la puesta en marcha del entorno de trabajo. Además, según la investigación realizada, aunque este tipo de redes neuronales ya existen, no se han encontrado casos de redes neuronales de superresolución que trabajen con datasets monocromáticos.

4.1. Entorno de trabajo

En este apartado, se va a llevar a cabo una descripción acerca de la máquina donde se va a poner en marcha la red neuronal, así como del entorno que se ha utilizado para su desarrollo y puesta en marcha.

Para empezar, la máquina utilizada cuenta con un procesador de 13ª generación de Intel, el Intel Core i9-13900k, que cuenta con 24 núcleos de procesamiento. Esto permite utilizar técnicas de paralelización para acelerar el tiempo de entrenamiento y aumentar la eficiencia de las redes neuronales entrenadas. Al tener más núcleos de procesador disponibles, la unidad central de procesamiento (en inglés, Central Processing Unit, CPU) puede procesar múltiples tareas simultáneamente, lo que acelera el tiempo de entrenamiento y reduce el tiempo necesario para obtener resultados precisos.

Además, la máquina tiene 4 módulos de memoria RAM DDR5 HYPERX, cada uno con una capacidad de 16 GB, lo que proporciona una memoria total de 64 GB.

La tarjeta gráfica MSI RTX 3070TI GAMING X TRIO de 8 GB GDDR6X es una tarjeta de alta gama diseñada específicamente para la computación en paralelo y el Deep learning.

Finalmente, el sistema operativo que se ha escogido ha sido Ubuntu 20.04.6 LTS, ya que proporciona una plataforma estable y fácil de usar para el desarrollo de aplicaciones de IA. Además, ofrece una amplia selección de herramientas y bibliotecas de software que son compatibles con los frameworks con los que se va a trabajar.

Para este TFG, se ha decidido aprovechar la unidad de procesamiento gráfico (en inglés *Graphics Processing Units*, GPU) para llevar a cabo el entrenamiento de las redes neuronales, ya que si se tiene una buena tarjeta gráfica, los resultados son mucho mejores en los entrenamientos con GPU que con CPU. Por lo tanto, la GPU se utiliza principalmente para realizar cálculos intensivos, de manera paralelizada, en el procesamiento de datos de entrenamiento de redes neuronales. La memoria RAM es necesaria para garantizar que la GPU tenga suficientes datos de entrenamiento para trabajar de manera eficiente y sin interrupciones, y además, la CPU será necesaria para el manejo general del sistema y la transferencia de datos entre la memoria RAM y la GPU. Mediante esta configuración, se pretende maximizar la eficiencia de la GPU durante el entrenamiento.



Figura 10. Máquina utilizada durante el entrenamiento de la red.

A continuación, se van a detallar las librerías seleccionadas, así como los pasos que se han seguido para configurarlas correctamente. Las librerías que se han seleccionado para implementar la red SRGAN, en Python, son Tensorflow y PyTorch, ya que permiten realizar modificaciones sobre los modelos de redes neuronales, así como permiten ser configuradas para aprovechar la GPU de la máquina.

Por un lado, **TensorFlow** ^[26] es una biblioteca de código abierto para la computación numérica, en Python, que permite construir y entrenar modelos de aprendizaje automático, en particular redes neuronales. Fue desarrollado por el equipo de Google Brain y se ha convertido en una de las herramientas más populares en el campo del aprendizaje profundo y la IA.

TensorFlow permite crear modelos de aprendizaje automático a través de un enfoque basado en gráficos, donde las operaciones matemáticas se representan como nodos en un grafo, y los datos fluyen a través de ellos en forma de tensores (arrays multidimensionales). Estos grafos son computados por el motor de TensorFlow, lo que permite una gran flexibilidad y escalabilidad en el diseño y entrenamiento de modelos complejos.

Además de ser una herramienta extremadamente poderosa para la creación de redes neuronales, facilita la creación de las mismas con diferentes niveles de abstracción. Tensorflow es capaz de utilizar el hardware disponible, de manera eficiente, lo que le permite aprovechar al

máximo las capacidades de las unidades de procesamiento gráfico para acelerar el entrenamiento de modelos.

Las GPUs son componentes hardware diseñados para realizar cálculos matemáticos complejos de manera rápida y eficiente. Originalmente fueron utilizados para mejorar el rendimiento de los gráficos en videojuegos y aplicaciones multimedia, pero ahora también se utilizan para acelerar el entrenamiento de modelos de aprendizaje automático. Las GPUs son capaces de realizar cálculos en paralelo, lo que las hace ideales para el entrenamiento de redes neuronales, ya que este proceso implica una gran cantidad de operaciones matemáticas repetitivas. Al utilizar las capacidades de las GPU, se puede acelerar significativamente el entrenamiento de modelos y mejorar su rendimiento.

Por otro lado, **PyTorch** ^[27] es una biblioteca de aprendizaje automático, de código abierto, basada en Python que se utiliza para el desarrollo de redes neuronales y otras aplicaciones de aprendizaje automático. Fue desarrollada por el equipo de investigación de IA de Facebook y se ha convertido en una de las bibliotecas más populares en el campo del aprendizaje profundo.

PyTorch permite crear y entrenar redes neuronales de manera eficiente y se ha convertido en una de las herramientas preferidas por los investigadores y desarrolladores de Deep Learning. Ofrece una interfaz sencilla y flexible para definir y manipular los tensores, que son los objetos básicos para representar los datos y los cálculos. Esto es especialmente útil en tareas de investigación en las que se necesitan cambios frecuentes en la arquitectura del modelo.

Además, PyTorch ofrece una amplia gama de herramientas para la visualización y el depurado de modelos, lo que facilita la comprensión y el ajuste de los mismos durante el proceso de desarrollo. También es compatible con GPU, lo que permite una aceleración significativa en el entrenamiento y la inferencia de modelos.

A continuación, se va a detallar el proceso de instalación para cada una de las librerías mencionadas, así como los pasos a realizar para que aprovechen al máximo las GPUs de una máquina.

4.1.1. Instalación y configuración en Tensorflow

Para llevar a cabo ^[28] la instalación y configuración de Tensorflow, se han seguido los siguientes pasos:

1. Se ha verificado que los requisitos del sistema se cumplen. Estos requisitos son los siguientes:
 - i. Ubuntu 16.04 o superior (16 bits).
 - ii. Python 3.9.
 - iii. El sistema gestor de paquetes de Python pip, con versión 19.0 o superior.
 - iv. Controladores de GPU NVIDIA con versión 450.80.02 o superior.

2. Una vez se han cumplido los requisitos anteriores, se ha procedido a instalar el gestor de paquetes Miniconda, el cual lleva integrado tanto Tensorflow como su configuración para GPU.

```
curl
https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -o Miniconda3-latest-Linux-x86_64.sh

bash Miniconda3-latest-Linux-x86_64.sh
```

3. Se ha creado el ambiente de trabajo para TensorFlow denominado 'tf'.

```
conda create --name tf
python=3.9
```

Para activarlo:

```
conda activate tf
```

4. Se ha configurado correctamente la GPU:
 - i. Primero, se ha verificado que el controlador gráfico esté instalado correctamente:

```
nvidia-smi
```

- ii. Luego, se ha instalado CUDA y cuDNN con conda y pip:

```
conda install -c conda-forge
cudatoolkit=11.8.0
```

```
pip install nvidia-cudnn-cu11==8.6.0.163
```

- iii. Se han configurado las rutas del sistema:

```
mkdir -p
$CONDA_PREFIX/etc/conda/activate.d
```

```
echo 'CUDNN_PATH=$(dirname
$(python -c "import
nvidia.cudnn;print(nvidia.cudnn.__file__)"))' >>
$CONDA_PREFIX/etc/conda/activate.d/env_vars.sh
```

```
echo 'export
LD_LIBRARY_PATH=$LD_LIBRARY_PATH:$CONDA_PREFIX/lib/:$CUDNN_PATH/lib' >>
$CONDA_PREFIX/etc/conda/activate.d/env_vars.sh
```

5. Se ha instalado TensorFlow.

```
pip install --upgrade pip
```

```
pip install tensorflow==2.12.*
```

6. Se ha verificado la instalación.

```
python3 -c "import tensorflow
as tf;
print(tf.reduce_sum(tf.random.
normal([1000, 1000])))"
```

```
python3 -c "import tensorflow
as tf;
print(tf.config.list_physical_
devices('GPU'))"
```

4.1.2. Instalación y configuración en PyTorch

En este caso, para llevar a cabo la instalación y configuración de PyTorch, se han seguido las instrucciones indicadas en el sitio web oficial ^[30] de la librería. En este sitio, se dan indicaciones sobre el sistema operativo (Linux, Mac o Windows), el gestor de paquetes con el que se iba a instalar (Pip, Conda o Source), el lenguaje de programación (Python o Java) y la plataforma de computación (CPU o GPU). ^[29]

PyTorch Build	Stable (2.0.0)		Preview (Nightly)	
Your OS	Linux	Mac	Windows	
Package	Conda	Pip	LibTorch	Source
Language	Python		C++ / Java	
Compute Platform	CUDA 11.7	CUDA 11.8	ROCm 5.4.2	CPU
Run this Command:				

Figura 11. Página de PyTorch para instalar su librería en local. ^[27]

En este TFG se va a llevar a cabo su instalación con una versión estable, en Linux (Ubuntu), mediante Pip, lenguaje de programación Python y se va a hacer uso de la GPU. La instrucción a ejecutar es la siguiente:

```
pip3 install torch torchvision torchaudio
```

Una vez instalado, se verifica que PyTorch funciona correctamente en la máquina mediante:

```
import torch
x = torch.rand(5, 3)
print(x)
```

De esta forma, si se devuelve un Tensor (una matriz) de tamaño 5 filas y 3 columnas con valores aleatorios, es que PyTorch está instalado correctamente.

Por otro lado, se ha verificado que el driver de la GPU estaba habilitado y accesible por PyTorch:

```
import torch
print(torch.cuda.is_available())
```

Si en este caso, se imprime en consola 'True', significará que Torch está disponible para ejecutar sus operaciones en la GPU. En caso contrario, imprimirá 'False'.

4.2. Preprocesamiento de los datos

En cuanto al dataset con el que se va a trabajar, se ha optado por un conjunto de imágenes ^[31] radiológicas de tórax. Concretamente, 5.863 imágenes de rayos X, donde hay imágenes tanto de tórax normales como con neumonía. Como se puede entender, el objetivo inicial de este dataset era la clasificación (Normal o neumonía), aunque como contiene licencia de adaptación, se va a llevar a cabo un preprocesamiento de las imágenes para adaptarlas a las necesidades de este TFG.

A continuación, se muestran algunas imágenes del dataset seleccionado para ilustrar el tipo de imágenes que se utilizarán durante el entrenamiento de la red neuronal. Estas imágenes son radiografías de tórax y muestran tanto casos normales como casos con neumonía. El preprocesamiento de estas imágenes permitirá adaptarlas a las necesidades del proyecto y utilizarlas para el entrenamiento y validación de la red SRGAN.

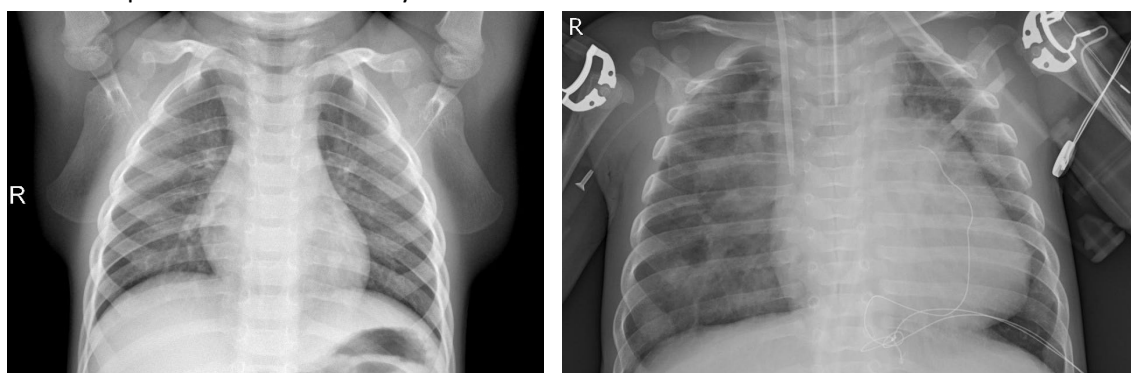


Figura 12. Imágenes del dataset seleccionado. ^[31]

Para empezar, el dataset contiene tres directorios, uno con el conjunto de imágenes de entrenamiento, otro con el conjunto de imágenes de validación, y un tercero con el conjunto de imágenes de test o prueba. En el directorio de entrenamiento, hay 1.341 imágenes de tórax normales y 3.875 imágenes de tórax con neumonía. En el directorio de validación, hay 8 imágenes de cada clasificación, y en el directorio de test, hay 234 imágenes de tórax normales y 390 imágenes de tórax con neumonía.

DIRECTORIO	NÚMERO DE IMÁGENES NORMALES	NÚMERO DE IMÁGENES CON NEUMONÍA
Entrenamiento	1.341	3.875
Validación	8	8
Test	234	390

Tabla 2. Tabla resumen del dataset. ^[31]

Como en este TFG no se va a realizar ninguna clasificación (el objetivo es mejorar las imágenes), se van a combinar las imágenes normales y de neumonía de cada directorio con el fin de ampliar el conjunto de imágenes. El conjunto de entrenamiento contiene imágenes necesarias durante el entrenamiento de la red SRGAN cuyo objetivo es ajustar los pesos de forma adecuada. Las imágenes de validación, son aquellas que se utilizan en la validación del entrenamiento. Es decir, durante el entrenamiento, cada ciertas iteraciones, se debe realizar una validación con el fin de verificar que el entrenamiento está yendo bien. Finalmente, el conjunto de test, son las imágenes con las que se prueba la red una vez ha terminado su entrenamiento.

A continuación, se ha considerado necesario crear un script para conseguir de cada imagen, una copia de la misma en baja resolución. De esta forma se tendrá para cada imagen original , su imagen correspondiente en baja resolución. Para ello, a continuación, se va a indicar mediante pseudocódigo la estructura del script que se ha realizado. Se puede consultar dicho script en el repositorio de ^[32] GitHub creado para este TFG.

```
Inicio

Función reescalaImágenesDirectorio(directorio, factorMultip)
  Para cada imagen en el directorio
    Abrir la imagen
    Calcular la nueva resolución de la imagen
    Reescalar la imagen con la nueva resolución
    Reemplazar la imagen con la nueva resolución
  FinPara
FinFunción

Definir directorios de imágenes: d_entrenamiento, d_validación, d_test

reescalaImágenesDirectorio(d_entrenamiento, 4)
reescalaImágenesDirectorio(d_validación, 4)
reescalaImágenesDirectorio(d_test, 4)

Fin
```

Figura 13. Pseudocódigo del script que permite reescalar las imágenes del dataset. ^[33]

En la figura 13, se muestra una simple función, que recibe el path del directorio de imágenes y el factor de multiplicación (el porcentaje de reducción de la resolución). En este TFG, se ha considerado oportuno reducir la resolución de las imágenes un 75%, es decir, dividir su resolución entre 4. En este script, se recorren todas las imágenes del directorio, donde para cada una de ellas, se determina su nueva resolución, y se reescala. Finalmente, se reemplaza la imagen con la nueva resolución.

De esta forma, ya se tendrían los conjuntos de imágenes tanto en alta como en baja resolución. El siguiente paso es realizar el preprocesamiento de las imágenes, con el fin de aumentar la cantidad y la variabilidad de los datos de entrenamiento, lo que puede mejorar la precisión y la capacidad de generalización del modelo. En este caso, el preprocesamiento va a ser diferente para la red implementada en Tensorflow y la red implementada en PyTorch, ya que como se verá posteriormente, PyTorch contiene un conjunto de funciones definidas que permiten realizar la carga de los imágenes aplicando un conjunto de transformaciones sobre las mismas.

4.3. Creación de la red neuronal en Tensorflow

En esta sección se va a llevar a cabo la implementación de la red neuronal en Tensorflow. Para ello, se va a comenzar indicando la red neuronal de la que se parte, las modificaciones que se han llevado a cabo sobre la misma para conseguir una red neuronal SRGAN que trabaje eficientemente con imágenes monocromáticas, y finalmente, se van a indicar los problemas que se han tenido durante el proceso.

4.3.1. Red neuronal de partida

La red neuronal de la que se ha partido, se encuentra en el siguiente directorio de GitHub: <https://github.com/jlaihong/image-super-resolution>, donde el código se encuentra repartido en otros 3 directorios diferentes: un directorio denominado 'datasets', que contiene los datasets a utilizar, y los scripts que permiten la carga y procesamiento de las imágenes, otro directorio denominado 'models', que contiene la arquitectura del generador y del discriminador, y el último directorio 'utils', que contiene ficheros python con funciones útiles como métricas, callbacks (permiten hacer checkpoints de los pesos entrenados de la red), y otras funciones de mapeo del dataset. Además, en el directorio raíz, está el fichero de entrenamiento, el cual importa el resto de ficheros, entrena y evalúa la red SRGAN. Para empezar a detallar esta arquitectura de red, se va a comenzar por el fichero principal, el de entrenamiento.

Este fichero es un cuaderno de Jupyter¹, que contiene el código para entrenar la arquitectura SRGAN definida para la superresolución de imágenes. Para empezar, comienza con la preparación del dataset, donde el pseudocódigo es el siguiente:

```
Inicio

Definir variables de configuración: dataset_key, data_path, hr_crop_size

Definir lista de transformaciones a aplicar sobre dataset de entrenamiento
- recorte aleatorio utilizando un tamaño de recorte (hr_crop_size) y el
  factor de multiplicación
- volteo aleatorio
- rotación aleatoria
- ruido aleatorio en la imagen de baja calidad

Definir conjuntos de datos de entrenamiento y validación a partir de
las variables y transformaciones anteriores

Sobre el conjunto de validación, tomar un subconjunto de imágenes

Fin
```

Figura 14. Pseudocódigo asociado a la preparación del dataset. ^[34]

Sobre el dataset original, se obtienen los conjuntos de datos de entrenamiento y de validación. En el caso del conjunto de entrenamiento, se aplican una serie de transformaciones con el fin de obtener un mayor número de imágenes diferentes y mayor variedad. Para ello, se van a aplicar transformaciones como: recortes (fragmentos aleatorios de las imágenes), volteos (donde se aplican de forma aleatoria reflexiones horizontales o verticales de las imágenes), rotaciones en un número aleatorio de grados, y la agregación de ruido de compresión JPEG a las imágenes. Sobre el dataset de validación, no se realiza ninguna transformación, únicamente se

¹ Jupyter es un software libre que permite crear y compartir documentos que contienen código, texto, gráficos y otros elementos. Soporta más de 40 lenguajes de programación, incluido Python.

obtiene un subconjunto de imágenes de las mismas, con el fin de acelerar las tareas de evaluación durante el entrenamiento.

A continuación, en el fichero se pasa a definir y entrenar el modelo del generador. En este caso, el creador de la red, considera que realizando un preentrenamiento sobre el generador, se asegura que el generador pueda obtener imágenes de alta resolución antes de comenzar el entrenamiento adversario con el discriminador, lo que puede mejorar la calidad de las imágenes generadas. Para ello, obtiene el modelo previamente definido, define los checkpoints, que son unas copias de seguridad de los pesos entrenados de la red, y comienza el entrenamiento del generador, donde se define como función de pérdida la métrica **Mean Squared Error** (MSE). Esta métrica, denominada como error cuadrático medio, es muy útil para determinar cuánto difiere la salida del modelo de la imagen real, ya que determina el error cuadrático medio entre los valores predichos por el modelo y los valores reales. Finalmente, al acabar con el entrenamiento, guarda en un fichero los pesos del generador.

```
Inicio

  Construye el modelo del generador

  Define las variables: checkpoint_dir, learning_rate

  Define el administrador de checkpoints, que guarda y carga los
  puntos de control
  Si quedan épocas por entrenar entonces
  - Compila el modelo con el optimizador y la función de pérdida
    MeanSquaredError
  - Entrena el modelo en el conjunto de datos de entrenamiento y
    validación
  FinSi
  Crea el directorio para guardar los pesos del modelo
  Guarda los pesos del modelo en un archivo .h5

Fin
```

Figura 15. Pseudocódigo asociado al preentrenamiento del generador. ^[34]

En cuanto a la arquitectura desarrollada en el fichero 'srresnet.py', ubicada en el directorio 'models', define la función build_srresnet que crea el modelo del generador. Para ello, sobre la entrada de la red, aplica una normalización, donde divide el valor de cada pixel de la imagen entre 255. Posteriormente, la imagen pasa a un conjunto de bloques residuales. Cada uno de estos bloques residuales está compuesto de dos capas convolucionales, cada una seguida de una capa de normalización por lotes (en inglés *Batch Normalization*) y una función de activación ReLU. La entrada a un bloque residual se pasa, a través de estas capas convolucionales, y se agrega a la salida de las mismas antes de pasarla a la siguiente capa. Posteriormente, el modelo define los bloques de upsample, donde el número de los mismos dependerá del factor de multiplicación. En este caso, al ser un factor de 4, habrá dos bloques de upsample, que serán los encargados de aumentar la resolución de la imagen mediante la técnica denominada "Pixel Shuffler x2".

En la siguiente figura 16, se va a llevar a cabo una representación del código del fichero en pseudocódigo:

```
Inicio

Función pixel-shuffle(imagen, factorMultip)
| Devuelve imagen con bloques de píxeles más grandes y detallados
FinFunción

Función upsample(imagen, numFiltros)
| Crea capa convolucional con numFiltros y tamañoKernel=3
| Llama función pixel-shuffle con factor=2
| Devuelve imagen tras aplicar función de activación PReLU
FinFunción

Función residual-block(imagen, numFiltros)
| Crea capa convolucional con numFiltros y tamañoKernel=3
| Aplica normalización por lotes
| Aplica función de activación PReLU
| Crea capa convolucional con numFiltros y tamañoKernel=3
| Aplica normalización por lotes
| Devuelve la imagen formada por la imagen de entrada y la imagen
| resultado de la normalización por lotes
FinFunción

Función build-srresnet(imagen)
| Define la capa de entrada
| Normaliza la imagen de entrada dividiendo cada pixel entre 255

| Para i in 0..num_bloques_residuales-1
| | Llama función residual-block
| FinPara

| Para i in 0..num_bloques_upsample-1
| | Llama función upsample
| FinPara

| Crea capa convolucional con numFiltros 3, tamañoKernel=9 y función
| de activación tanh.

| Denormaliza los valores de los píxeles sumando 1 y multiplicando
| por 127.5
FinFunción

Fin
```

Figura 16. Pseudocódigo asociado a la definición del modelo del generador. ^[35]

En cuanto a la arquitectura que se define sobre el discriminador, se trata de la misma que se ha definido anteriormente.

Finalmente, quedaría el entrenamiento combinado, es decir, el entrenamiento formado conjuntamente por el generador y el discriminador. Para ello, previamente se cargan los pesos entrenados del generador y se obtiene el modelo del discriminador. Finalmente, se procede a entrenar la red durante un número determinado de ciclos y épocas de entrenamiento (ver página 8).

Durante el entrenamiento, se van entrenando tanto la red generadora como la discriminadora, y cada 1000 ciclos de entrenamiento, se realiza la validación del mismo.

A continuación, se va a llevar a cabo una representación del código del entrenamiento en pseudocódigo:

```
Inicio

Construye el modelo del generador
Carga los pesos entrenados previamente del generador

Construye el modelo del discriminador

Importa y construye el modelo perceptual asociado a la red VGG-19

Define el directorio de checkpoints
Si existe checkpoint entonces recupera el entrenamiento

Función train-step(imagenBajaRes, imagenAltaRes)
| sr = generador(imagenBajaRes)

| srOutput = discriminador(imagenGenerada)
| hrOutput = discriminador(imagenAltaRes)

| mse = calculate-content-loss(hrOutput, srOutput)
| gen_loss = calculate-generator-loss(srOutput)
| perc_loss = mse + 0.001 * gen_loss
| disc_loss = calculate-discriminator-loss(hrOutput, srOutput)

| Se aplican los gradientes al generador y al discriminador

| Devuelve perc-loss, disc-loss
FinFunción

Función calculate-content-loss(hrOutput, srOutput)
| sr_features = vgg19(srOutput)
| hr_features = vgg19(hrOutput)

| Devuelve MSE(hr_features, sr_features)
FinFunción

Función calculate-generator-loss(srOutput)
| Devuelve binary_cross_entropy(matrizDeUnos, srOutput)
FinFunción

Función calculate-discriminator-loss(hrOutput, srOutput)
| hr_loss = binary_cross_entropy(matrizDeUnos, hrOutput)
| sr_loss = binary_cross_entropy(matrizDeCeros, srOutput)

| Devuelve (hr_loss + sr_loss)
FinFunción
```

```

Para lr, hr in train_dataset:
    perceptual_loss, discriminator_loss = train_step(lr, hr)

    Si step % 1000 == 0 entonces
        Para lr, hr in valid_dataset:
            sr = generator(lr)
            psnr = psnr_metric(hr, sr)
        FinPara

        Guarda la imagen generada en formato .png

        Muestra por consola el estado del entrenamiento
    FinSi
FinPara
Fin

```

Figura 17. Pseudocódigo asociado al entrenamiento de la SRGAN. ^[34]

De esta forma, quedaría definida la arquitectura de la red de la que se ha partido para el desarrollo llevado a cabo por Tensorflow.

4.3.2. Modificaciones realizadas

En este apartado, se van a detallar las modificaciones que se han realizado sobre el código original de la red neuronal. Cabe indicar que todo el código propio, junto con las imágenes utilizadas y los resultados obtenidos, se pueden consultar en el repositorio ^[32] mencionado previamente.

Para poder alcanzar el objetivo planteado, se han realizado diferentes modificaciones con el fin de adaptar el código al caso de estudio de este TFG, el procesamiento de imágenes monocromáticas. Estas modificaciones se van a resumir en la siguiente tabla:

	ORIGINAL	MODIFICADO
Decodificación de imágenes	decode_png(channels=3)	decode_jpeg(channels=3)
Arquitectura del generador	Input(shape=(None, None, 3))	Input(shape=(None, None, 1))
Arquitectura del discriminador	Input(shape=(None, None, 3))	Input(shape=(None, None, 1))
Adaptación de VGG-19	Input(shape=(None, None, 3))	Concatenate()([img, img, img])

Tabla 3. Modificaciones realizadas en Tensorflow para el procesamiento monocromático

También ha sido necesaria la adaptación de la red VGG-19, ya que al tratarse de una red pre-entrenada, ésta espera recibir imágenes policromáticas. Esta es la razón por la que se ha concatenado la misma imagen tres veces con el fin de tener las imágenes de entrada con tres canales. Una imagen puede estar formada por 3 canales porque el modelo de color RGB usa los tres colores primarios (rojo, verde y azul) para formar imágenes a través de rayos luminosos o píxeles.

A continuación, se va a explicar el trabajo llevado a cabo con los **hiperparámetros**² (ver página 11) de la red con el fin de obtener los valores óptimos para cada uno de ellos.

En este caso, los hiperparámetros que se han ajustado son: el tamaño de batch, el tamaño de recorte, el número de filtros de la red generadora y el número de bloques residuales. Para todas las comparativas realizadas, se va a aplicar la raíz del error cuadrático medio (en inglés, *Root Mean Squared Error*, **RMSE**), ya que expresa la diferencia relativa entre los valores de los píxeles de la imagen generada y los valores de los píxeles de la imagen original.

Se va a comenzar por el **tamaño de batch** (en inglés, *batch size*), el cual define el número de muestras que se introducen en la red para que sean procesadas simultáneamente. Si este valor es pequeño, significa que la red tiene en memoria pocas imágenes, y por lo tanto el entrenamiento es más rápido. Sin embargo, es posible que no se trabajen adecuadamente las características y detalles de las imágenes, y por lo tanto, el resultado sea peor. Por otro lado, si este valor es grande, el entrenamiento será más lento, pero la red será capaz de extraer un conjunto de características más amplio de cada imagen.

Los valores que se suelen utilizar para este hiperparámetro son potencias de 2, comenzando desde un tamaño de batch de 1, es decir, una única imagen en cada entrenamiento, hasta tamaños de 8, 16, 32, 64, etc. En este caso, se ha probado como tamaño de batch 1, 8, 16 y 32, y el resultado ha sido el siguiente:

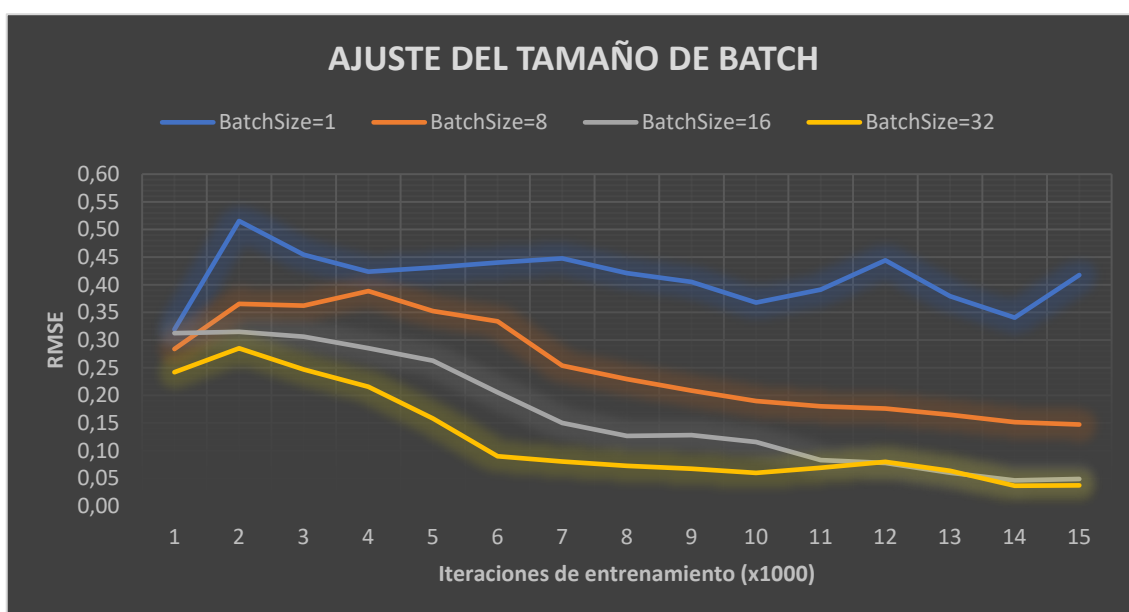


Figura 18. Ajuste del tamaño de batch en Tensorflow

Mediante la figura anterior, se puede observar que aumentando el tamaño de batch, para un mismo número de iteraciones (concretamente 15.000), se obtiene un RMSE más bajo, y por tanto, se generan imágenes más realistas. Cabe indicar que tanto para un tamaño de batch de 16 como de 32, el RMSE que se logra tras 15.000 iteraciones es el mismo, y por lo tanto, se ha decidido ajustar el tamaño de batch a 16, ya que el tiempo de entrenamiento es inferior.

² Hiperparámetros en Tensorflow: <https://youtu.be/leXkkQU3QsM> [Último acceso: 06/06/2023].

El segundo hiperparámetro a ajustar es el **tamaño de recorte** (en inglés, *crop size*). Este valor indica la dimensión de las imágenes de entrenamiento. En la fase de carga y preprocesamiento de imágenes en Tensorflow, para el conjunto de entrenamiento, se aplican una serie de transformaciones, y una de ellas es el recorte aleatorio. Esta transformación consiste en recortar una región aleatoria de cada imagen, y utilizar ese recorte como nueva imagen de entrenamiento, sustituyendo a la imagen anterior. Con esto, se consigue que las imágenes que van a ser procesadas por la red sean más pequeñas, y por lo tanto, al haber menos información, el entrenamiento sea más rápido.

Es importante elegir un tamaño de recorte adecuado. Un tamaño demasiado pequeño puede perder detalles importantes de las imágenes de entrada, lo que puede afectar negativamente la calidad de la superresolución. Por otro lado, un tamaño de crop demasiado grande puede requerir más memoria y poder computacional.

El rango de valores que se puede utilizar para este hiperparámetro, lo limita el tamaño de la imagen original, es decir, que como máximo, el recorte podrá tener una dimensión igual a la de la imagen. En este caso, se han probado tamaños de recorte de 24, 48, 96 y 192. A continuación, se comparan los resultados obtenidos para 24, 48 y 96.

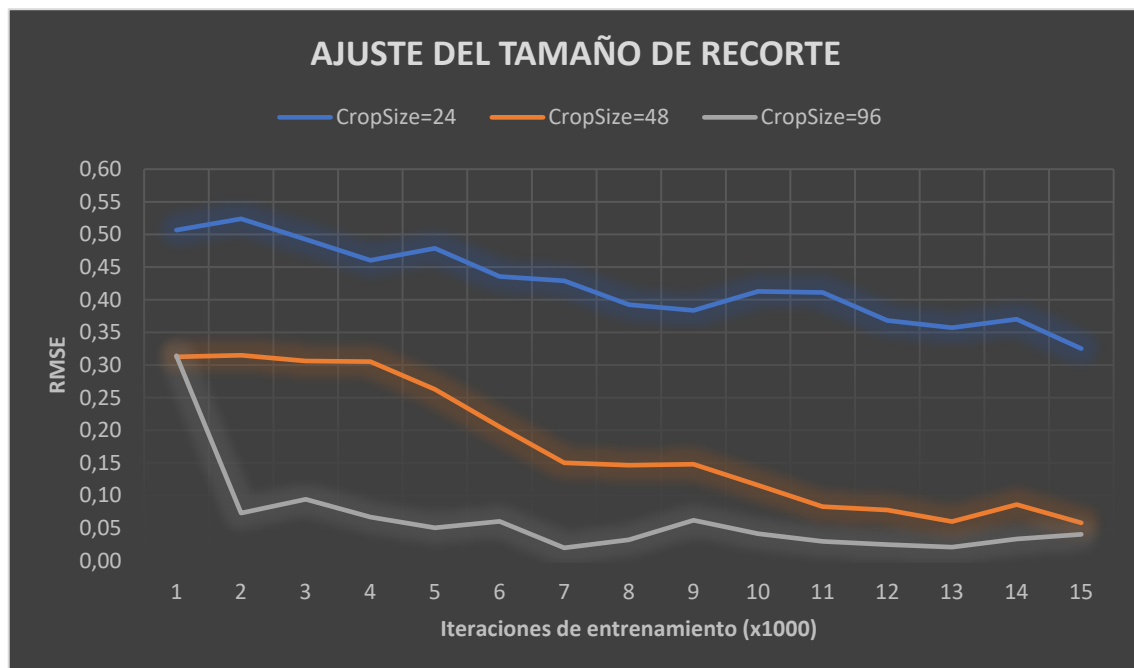


Figura 19. Ajuste del tamaño de recorte en Tensorflow

En el caso de un tamaño de recorte de 192, provocaba al entrenamiento una finalización inmediata debido a falta de memoria, y por lo tanto, no se han obtenido resultados del mismo. En cuanto a los resultados del gráfico anterior, se puede observar que a partir de un tamaño de recorte de 48, ya se obtienen resultados deseables para el RMSE. En este caso, en 15.000 iteraciones, se ha logrado un RMSE del 4%. Además, la diferencia que existe al utilizar recortes de 48x48 o de 96x96 es leve, y por lo tanto, se ha considerado utilizar un recorte de tamaño 48x48.

El siguiente hiperparámetro a ajustar es el **número de filtros** a utilizar en la red generadora. El generador es una parte fundamental de la red SRGAN, ya que se encarga de generar imágenes de superresolución a partir de las imágenes de baja resolución. El número de filtros del generador es un hiperparámetro que determina la complejidad y capacidad del modelo para capturar detalles y características de las imágenes.

Los filtros del generador son capas convolucionales que se utilizan para extraer características de las imágenes de baja resolución y transformarlas en características de alta resolución. Estos filtros actúan como detectores de patrones y se organizan en capas para capturar diferentes niveles de detalle.

El número de filtros determina la cantidad de características y detalles que el modelo puede aprender. Un número mayor de filtros permite al generador capturar características más finas y sutiles en las imágenes generadas. Sin embargo, un número demasiado alto de filtros puede aumentar la complejidad del modelo y requerir más recursos computacionales sin obtener mejoras notables en el resultado.

El rango de valores posibles para el número de filtros del generador varía según la arquitectura específica de la red y el tamaño de las imágenes de entrada. En general, los valores típicos se encuentran en un rango entre 16 y 512 filtros. En este caso, se han probado los valores 32, 64 y 128 para el número de filtros, y se han obtenido los siguientes resultados:

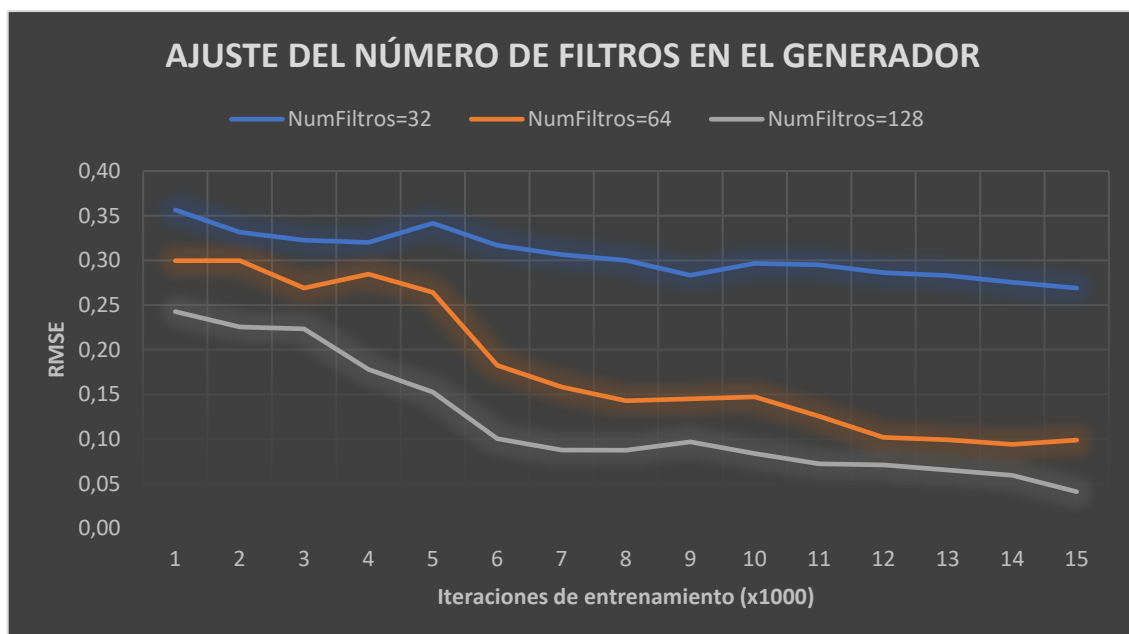


Figura 20. Ajuste del número de filtros del generador en Tensorflow

Cuando se utilizan 64 o 128 filtros, se obtiene un valor similar de RMSE al cabo de las 15.000 iteraciones. Es por ello por lo que, para unos resultados semejantes, se ha escogido el número de filtros que tiene un menor tiempo de entrenamiento. En este caso, el valor seleccionado para el número de filtros en la red generadora ha sido de 64.

Por otro lado, está el hiperparámetro asociado al **número de bloques residuales** que contiene la red generadora.

Los bloques residuales son una parte fundamental de muchas arquitecturas de redes neuronales, incluidas las SRGAN. Estos bloques se utilizan para facilitar el entrenamiento profundo y mejorar la capacidad de la red para preservar y transmitir características importantes de las imágenes.

En el contexto de un SRGAN, los bloques residuales se componen de capas convolucionales y capas de normalización, como la normalización por lotes. Estos bloques están diseñados para permitir que la información fluya a través de la red sin ser alterada significativamente, lo que ayuda a preservar los detalles y características de las imágenes durante el proceso de superresolución.

El número de bloques residuales es un hiperparámetro que determina la profundidad de la red y la capacidad de capturar detalles finos en las imágenes generadas. Un número mayor de bloques residuales permite que la red capture características más sutiles y complejas, pero también aumenta la complejidad del modelo y la cantidad de parámetros que se deben aprender.

El rango de valores posibles para el número de bloques residuales puede variar según la arquitectura específica del SRGAN y la complejidad del problema. En general, los valores típicos se encuentran en un rango entre 5 y 20 bloques residuales. En este caso, se han probado un número de 5, 8 y 16 bloques residuales, y se han obtenido los siguientes resultados:

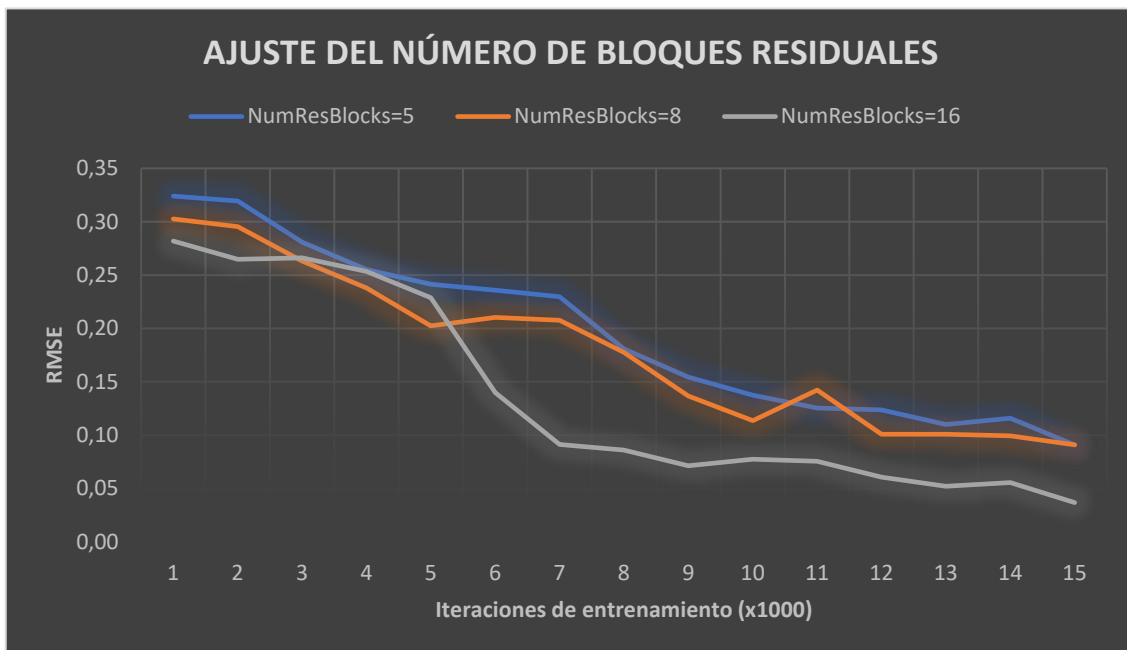


Figura 21. Ajuste del número de bloques residuales del generador en Tensorflow

Los resultados obtenidos para un número de bloques residuales igual a 5 y 8 son similares. Sin embargo, cuando se utilizan 16 bloques residuales, los resultados son mejores, y por lo tanto, ha sido el valor seleccionado.

4.3.3. Problemas encontrados

En esta sección, se van a mencionar diferentes problemas que se han tenido durante las modificaciones y la ejecución de la red implementada en Tensorflow.

Para empezar, el primer problema que se ha tenido ha sido la correcta configuración del entorno para **utilizar eficientemente la tarjeta gráfica**. Para ello, como se ha comentado en el apartado 4.2.1 (Instalación y configuración en Tensorflow), ha sido necesaria una instalación limpia de los controladores gráficos y de la librería de cómputo paralelo CUDA. Instalar ambas, que fueran compatibles entre ellas, y con el sistema operativo de la máquina, ha sido una de las principales dificultades. Una vez se ha tenido el entorno de Tensorflow configurado correctamente, las ventajas en los tiempos de entrenamiento, han sido muy notables.

A continuación, se presenta una gráfica donde se compara el número de iteraciones de entrenamiento realizadas en función del tiempo, tanto en un entorno donde se aprovecha la GPU, como en un entorno donde no está preparado para ello, y que por tanto, todas las operaciones se realizan en la CPU.

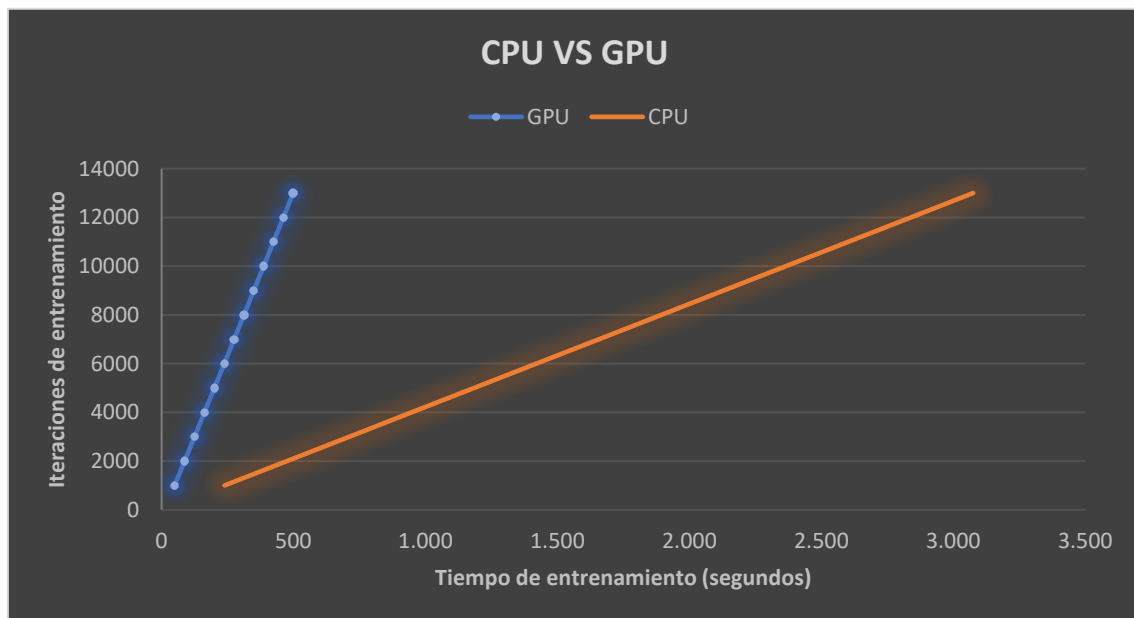


Figura 22. Comparativa del RMSE en el entorno Tensorflow con operaciones en CPU y GPU.

En este caso, para un entrenamiento de 14.000 iteraciones, realizado en el entorno de GPU y CPU, para el primer caso, se ha completado en 498 segundos, mientras que en el entorno de CPU, el entrenamiento se ha realizado en 3.000 segundos. Esto es debido a la capacidad de procesamiento paralelo que ofrece la GPU en comparación con la CPU.

Además, durante el entrenamiento de la red en un entorno configurado con GPU, se ha hecho uso del comando 'nvidia-smi' en la consola, útil para monitorizar la GPU del sistema, con el fin de obtener el porcentaje de uso de la misma durante el entrenamiento. La tarjeta gráfica tenía un 99% de 'GPU-Util', que quiere decir que está siendo utilizada al máximo de su capacidad durante el proceso de entrenamiento.

El segundo problema a abordar ha sido el **ajuste de los hiperparámetros** de la red neuronal.

En el caso del **número de filtros** de la red generadora, además de los valores que se han comentado anteriormente (32, 64 y 128), también se ha probado con 256 filtros, lo que ha provocado que genere imágenes con características indeseables. Aumentar el número de filtros puede aumentar la capacidad del modelo para capturar detalles finos, pero también puede hacer que el generador sea demasiado complejo y susceptible al sobreajuste.

El sobreajuste ocurre cuando el modelo se ajusta demasiado a los datos de entrenamiento y tiene dificultades para generalizar a datos nuevos. En el contexto de una red SRGAN, esto significa que el generador puede aprender a generar imágenes de superresolución que se asemejan demasiado a las imágenes de entrenamiento, pero no capturan adecuadamente las características generales de las imágenes de alta resolución.

A continuación, se presentan algunas imágenes que se han obtenido durante el entrenamiento de la red con un número de filtros igual a 256, donde se puede observar que aparecen características que realmente las imágenes originales no tienen:

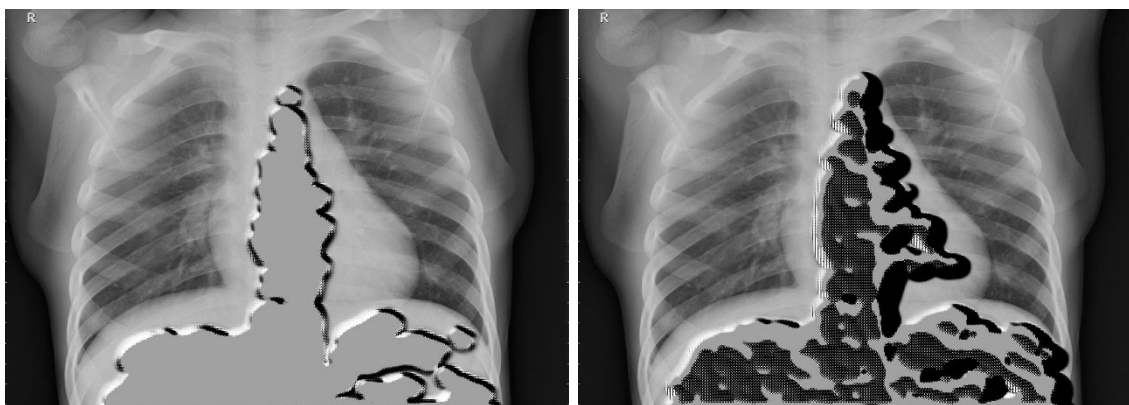


Figura 23. Resultado de utilizar un número de filtros muy grande.

El tercer y último problema ha surgido con la **tasa de aprendizaje** (en inglés, *Learning Rate* LR), que es el factor que controla cuánto se ajustan los pesos de la red durante el proceso de aprendizaje. Una tasa de aprendizaje baja implica un aprendizaje lento, pues se requerirán muchas iteraciones para alcanzar un mínimo. Por otro lado, una tasa de aprendizaje elevada implica una mayor inestabilidad en el algoritmo, que da saltos mucho más grandes, lo que puede significar que nunca se alcance la convergencia. Por lo tanto, el reto es encontrar el valor óptimo de la tasa de aprendizaje.

El rango de valores posibles para el LR varía según la arquitectura de la red, el conjunto de datos y el optimizador utilizado. A menudo, los valores típicos se encuentran en un rango entre 0,00001 ($1e-5$) y 0,1.

Durante el entrenamiento de la red, se han ido probando diferentes valores para la tasa de aprendizaje, concretamente: 0,00001 ($1e-5$); 0,0001 ($1e-4$) y 0,001 ($1e-3$).

Durante su ajuste, se han podido encontrar los siguientes problemas: cuando se definía un valor de 0,001 (una tasa de aprendizaje alta), las imágenes que producía el generador eran completamente negras, mientras que cuando se definía un valor de 0,00001 (una tasa de aprendizaje baja), las imágenes producidas por el generador no presentaban mejoras en cada validación. De estos resultados, se concluía que un valor medio entre ambas podría lograr la convergencia deseada. Es por ello por lo que la tasa de aprendizaje seleccionada es de 0,0001.

A continuación, se va a presentar mediante una comparativa, la imagen generada con una tasa de aprendizaje alta (imagen izquierda), y la imagen generada con una tasa de aprendizaje óptima (imagen derecha):

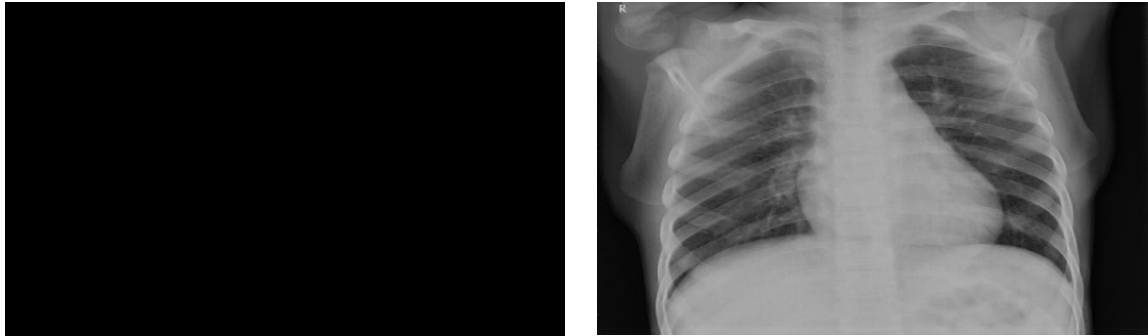


Figura 24. Comparativa de resultados al utilizar un LR alto y un LR óptimo.

Por otro lado, la imagen generada con una tasa de aprendizaje baja (imagen izquierda), y la imagen generada con una tasa de aprendizaje óptima (imagen derecha) serían las siguientes:

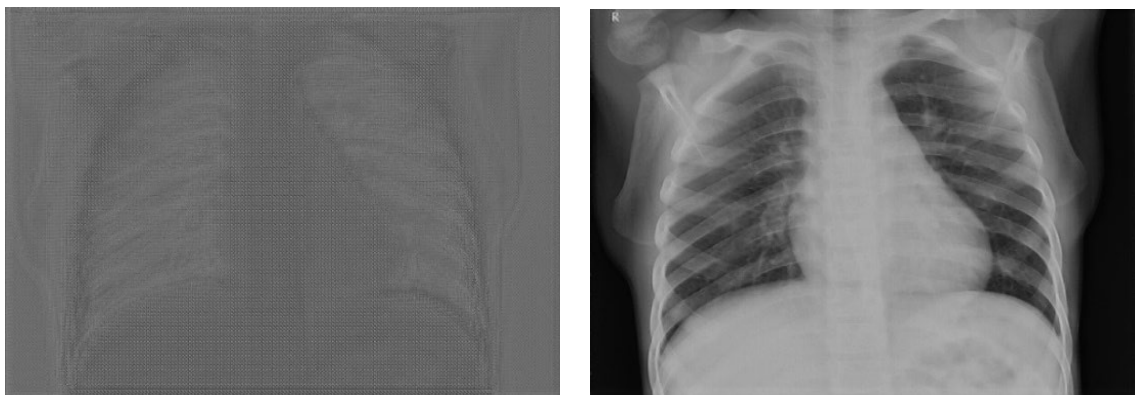


Figura 25. Comparativa de resultados al utilizar un LR bajo y un LR óptimo.

4.4. Creación de la red neuronal en PyTorch

Al igual que en el apartado anterior, se va a comenzar indicando la red neuronal de la que se parte, las modificaciones que se han llevado a cabo sobre la misma, para conseguir una red neuronal SRGAN que trabaje eficientemente con imágenes monocromáticas, y finalmente, se van a indicar los problemas que se han encontrado en su desarrollo.

4.4.1. Red neuronal de partida

La red neuronal de la que se ha partido, se encuentra en el siguiente directorio de GitHub: <https://github.com/eriklindernoren/PyTorch-GAN/tree/master/implementations/srgan>, donde, el código está repartido en 3 directorios diferentes. El primer directorio denominado 'datasets', contiene las funciones asociadas a la carga y procesamiento de las imágenes. El segundo directorio, denominado 'models', contiene la arquitectura del generador y del discriminador, y el último directorio denominado 'srgan', contiene las funciones asociadas al entrenamiento, el cual importa el resto de ficheros, entrena y evalúa la red SRGAN. Para empezar a detallar esta arquitectura de red, se van a explicar previamente los ficheros datasets y models, y se acabará comentando el fichero asociado al entrenamiento de la arquitectura.

El fichero 'dataset' define una clase denominada ImageDataset, la cual contiene una serie de funciones que permiten obtener y procesar las imágenes simultáneamente. En este caso, a partir de las imágenes originales, se obtienen tanto las imágenes en alta como en baja resolución. Para cada imagen, se aplican una serie de transformaciones que son: la redimensión de la imagen y la normalización de sus valores en un rango comprendido entre 0 y 1.

```
Inicio

Define para cada canal de las imágenes la media y desviación estándar

Clase ImageDataset(Dataset)

    Función lr-transform(img)
        Aplica transformación de redimensionamiento dividiendo anchura y altura
        entre 4
        Normaliza valores en rango entre 0 y 1
    FinFunción

    Función hr-transform(img)
        Aplica transformación de redimensionamiento
        Normaliza valores en rango entre 0 y 1
    FinFunción

Fin
```

Figura 26. Pseudocódigo asociado a las funciones de gestión del dataset ^[36]

La **normalización** de la información ayuda a asegurar que las imágenes de entrada tengan una escala coherente, lo que puede mejorar la precisión del modelo. Además, también puede ayudar a evitar que el modelo se vea afectado por datos atípicos y hacer que el proceso de entrenamiento sea más rápido y estable.

En cuanto a la definición de los modelos que componen la arquitectura SRGAN, definidos en el fichero 'models', establecen el modelo extractor de características VGG-19, el modelo asociado al generador, y el modelo asociado al discriminador de la red. La arquitectura de esta red, es exactamente la misma que en la red desarrollada en Tensorflow, salvo que en ésta, se utilizan las funciones asociadas a cada una de las capas correspondientes de PyTorch.

Por último, está el código asociado al entrenamiento de la red. Para ello, se obtienen los tres modelos de redes, el generador, el discriminador y la red VGG-19, y se recorren todas las imágenes del dataset x épocas de entrenamiento. Para cada imagen, se obtiene su imagen en super resolución, se obtiene la pérdida de contenido, y se propaga hacia atrás, y posteriormente, se entrena el discriminador, donde se obtienen las pérdidas del discriminador (error en la predicción con imagen generada y con imagen real), y nuevamente se propagan hacia atrás los errores con el fin de actualizar los pesos. Finalmente, cada 500 iteraciones, muestra por pantalla el estado del entrenamiento.

4.4.2. Modificaciones realizadas

En este apartado, se van a detallar las modificaciones que se han realizado sobre el código original de la red neuronal en PyTorch.

Para empezar, se han realizado diferentes modificaciones con el fin de adaptar el código al caso de estudio de este TFG, el procesamiento de imágenes monocromáticas. Estas modificaciones, se van a resumir en la siguiente tabla resumen:

	ORIGINAL	MODIFICADO
Decodificación de imágenes	-	<code>transforms.Grayscale(num_output_channels=1)</code>
Arquitectura del generador	<code>Generator(in_chann=3, out_chann=3)</code>	<code>Generator(in_chann=1, out_chann=1)</code>
Adaptación de VGG-19	<code>feature_extractor(img)</code>	<code>feature_extractor(torch.cat([img, img, img], dim=1))</code>

Tabla 4. Modificaciones realizadas en PyTorch para el procesamiento monocromático

En cuanto a la adaptación de la red VGG-19, también ha sido necesaria en este caso, ya que al tratarse de una red pre-entrenada, ésta espera recibir imágenes policromáticas. Es por ello, que se ha concatenado la misma imagen tres veces con el fin de tener las imágenes de entrada con tres canales.

A continuación, se va a explicar el trabajo llevado a cabo con los **hiperparámetros**³ de la red con el fin de obtener los valores óptimos para cada uno de ellos, en este caso, en PyTorch.

Los hiperparámetros que se han ajustado, al igual que en Tensorflow, son: el tamaño de recorte, el tamaño de batch, el número de filtros de la red generadora y el número de bloques residuales.

Se va a comenzar por el **tamaño de recorte** (en inglés, *crop size*). Este valor indica la transformación de dimensión de las imágenes de entrenamiento. Consiste en recortar una región aleatoria de cada imagen, y utilizar ese recorte como nueva imagen de entrenamiento, sustituyendo a la imagen anterior.

³ Hiperparámetros en PyTorch: <https://youtu.be/xP9l9MptIZo> [Último acceso: 06/06/2023].

El rango de valores que se puede utilizar para este hiperparámetro, lo limita el tamaño de la imagen original, es decir, que como máximo, el recorte podrá tener una dimensión igual a la de la imagen. En este caso, se han probado tamaños de recorte de 48, 96 y 250. En Tensorflow se ha probado con un tamaño de 192, y en este caso, se ha probado con un tamaño superior, de 250. Esto es debido a que en Tensorflow, en cada aumento de tamaño se ha podido confirmar la disminución del RMSE, sin embargo, en PyTorch no, y es por ello por lo que se ha probado con recortes más grandes. En la figura 27, se comparan los resultados obtenidos para los mismos:

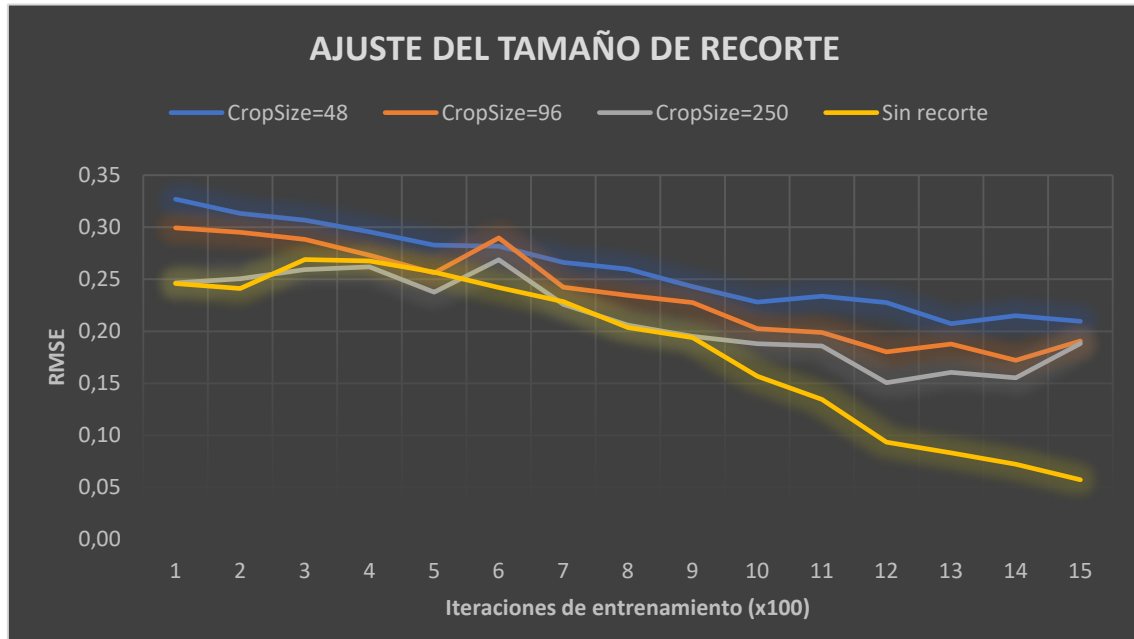


Figura 27. Ajuste del tamaño de recorte en PyTorch

Se ha considerado relevante añadir los resultados del RMSE obtenido sin aplicar ninguna transformación a las imágenes de entrenamiento. El entrenamiento es más lento, pero los resultados son mejores que si se realizan recortes sobre el conjunto de datos. Como se puede observar, en lugar de realizar 15.000 iteraciones de entrenamiento, en este caso, se han realizado 1.500 iteraciones, ya que al utilizarse imágenes más grandes, el tiempo de entrenamiento es superior, y por tanto, las validaciones se realizan cada menos iteraciones. Es por ello por lo que el hiperparámetro asociado al tamaño de recorte no va a ser ajustado.

A continuación, se va a determinar el **tamaño de batch** (en inglés, *batch size*), el cual define el número de muestras que se introducen en la red para que sean procesadas simultáneamente.

Los valores que se suelen utilizar para este hiperparámetro, son potencias de 2, comenzando desde un tamaño de batch de 1, es decir, una única imagen en cada entrenamiento, hasta tamaños de 8, 16, 32, 64, etc.

El hecho de que no se vayan a realizar recortes sobre las imágenes, afecta directamente al tamaño de batch, ya que las imágenes que hay cargadas en memoria son del tamaño original, y por lo tanto, si se intentan cargar varias imágenes simultáneamente, puede desembocar en una falta de memoria en el sistema.

En este caso, se han realizado pruebas para tamaño de batch igual a 1 y 2, y se ha podido comprobar que con un tamaño de batch igual a 1, la red ha generado imágenes correctas, mientras que cuando se ha aumentado el tamaño de batch a 2, la ejecución ha finalizado, indicando que el entrenamiento se ha quedado sin memoria, tal y como se adjunta a continuación:

```

OutOfMemoryError                                Traceback (most recent call last)
Cell In[7], line 25
    22 imgs_lr = Variable(imgs["lr"].type(Tensor))
    23 imgs_hr = Variable(imgs["hr"].type(Tensor))
--> 25 perceptual_loss, hr_loss, sr_loss, discriminator_loss = train_step(imgs_lr, imgs_hr)
    27 step = step + 1
    29 if step % sample_interval == 0:
    30     # Evaluate on validation set

```

Figura 28. Error por falta de memoria con un tamaño de batch superior a 1.

El siguiente hiperparámetro que se ha considerado modificar, al igual que en Tensorflow, es el **número de filtros** de la red generadora. El número de filtros del generador es un hiperparámetro que determina la complejidad y capacidad del modelo para capturar detalles y características de las imágenes.

Los filtros del generador son capas convolucionales que se utilizan para extraer características de las imágenes de baja resolución y transformarlas en características de alta resolución. Estos filtros actúan como detectores de patrones y se organizan en capas para capturar diferentes niveles de detalle.

El rango de valores posibles para el número de filtros del generador varía según la arquitectura específica de la red y el tamaño de las imágenes de entrada. En general, los valores típicos se encuentran en un rango entre 16 y 512 filtros. En este caso, se han probado los mismos valores que en Tensorflow: 32, 64 y 128. Con ellos, se han obtenido los siguientes resultados:

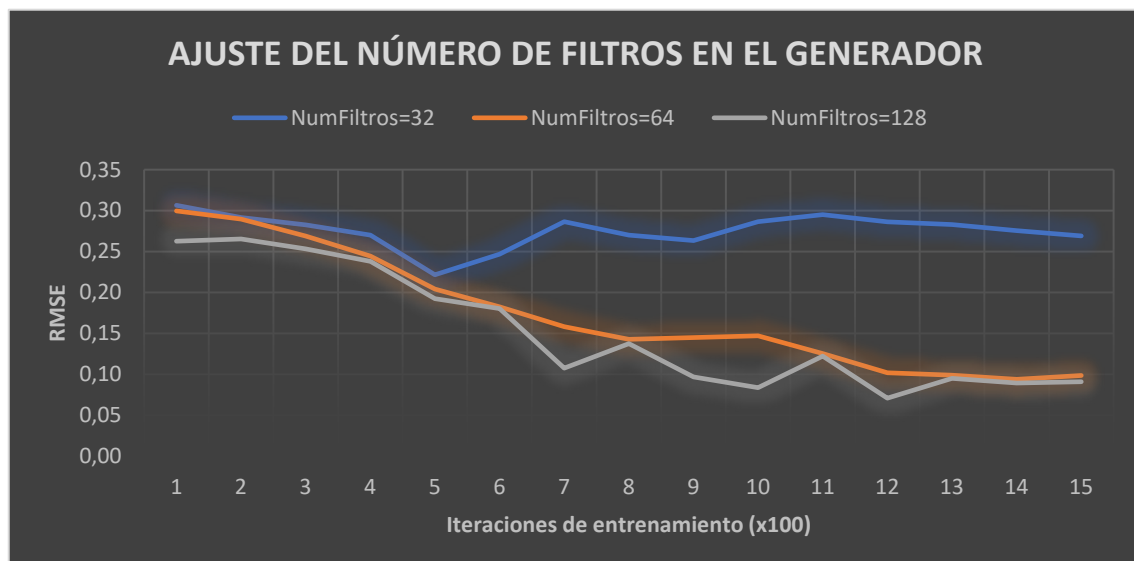


Figura 29. Ajuste del número de filtros en el generador en PyTorch

Nuevamente, cuando se utilizan 64 o 128 filtros, se obtiene un valor similar de RMSE al cabo de las 15.000 iteraciones. Por lo que, para unos resultados semejantes, se ha escogido el número de filtros que tiene un menor tiempo de entrenamiento. En este caso, el valor seleccionado para el número de filtros en la red generadora ha sido de 64.

Por último, está el hiperparámetro asociado al **número de bloques residuales** que contiene la red generadora.

Los bloques residuales son una parte fundamental de muchas arquitecturas de redes neuronales, incluidas las SRGAN. Estos bloques se utilizan para facilitar el entrenamiento profundo y mejorar la capacidad de la red para preservar y transmitir características importantes de las imágenes.

En el contexto de un SRGAN, los bloques residuales se componen de capas convolucionales y capas de normalización, como la normalización por lotes. Estos bloques están diseñados para permitir que la información fluya a través de la red sin ser alterada significativamente, lo que ayuda a preservar los detalles y características de las imágenes durante el proceso de superresolución.

El rango de valores posibles para el número de bloques residuales puede variar según la arquitectura específica del SRGAN y la complejidad del problema. En general, los valores típicos se encuentran en un rango entre 5 y 20 bloques residuales. En este caso, se han probado un número de 5, 8 y 16 bloques residuales, y se han obtenido los siguientes resultados:

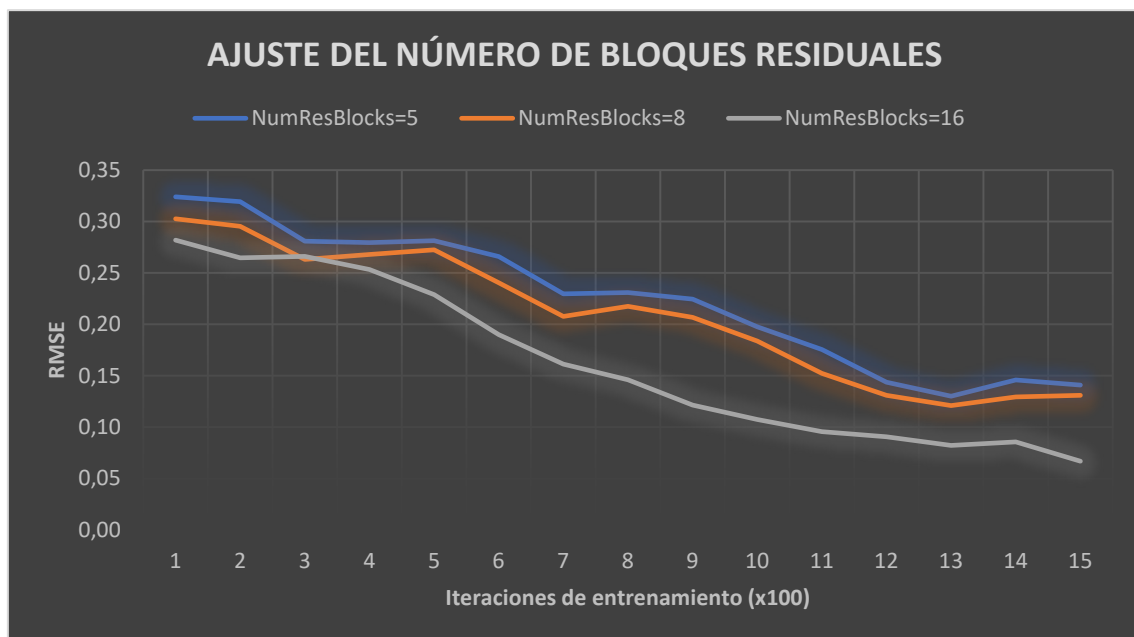


Figura 30. Ajuste del número de bloques residuales en PyTorch

Como se puede observar, los resultados obtenidos para un número de bloques residuales igual a 5 y 8 son similares. Sin embargo, cuando se utilizan 16 bloques residuales, los resultados son mejores, y por lo tanto, ha sido el valor seleccionado.

4.4.3. Problemas encontrados

A continuación, se van a mencionar diferentes problemas que se han tenido durante las modificaciones y la ejecución de la red implementada en PyTorch.

El primer problema ha vuelto a ser con la correcta **configuración del entorno**, en concreto, con la tarjeta gráfica de la máquina. Al igual que en el entorno anterior, se han realizado unas mediciones acerca de la mejora en tiempos de entrenamiento mediante el uso de la GPU en lugar de CPU. En la siguiente figura, se puede visualizar dicha mejora:

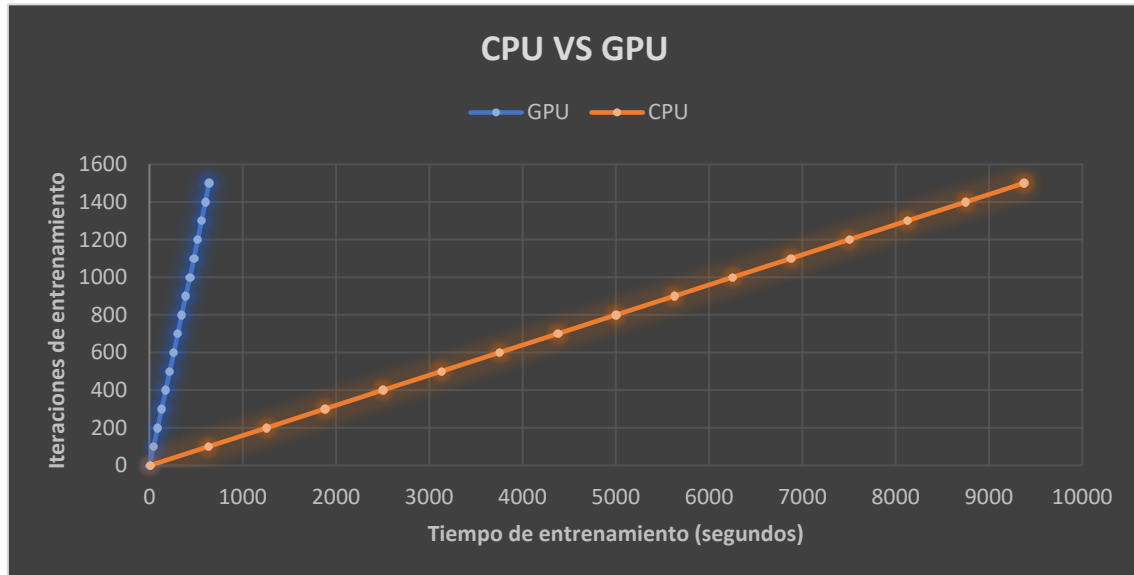


Figura 31. Comparativa del RMSE en el entorno PyTorch con operaciones en CPU y GPU.

Como se puede observar en el gráfico, para un entrenamiento de 1.500 iteraciones, cuando se ha hecho uso de la GPU, se ha completado en 638 segundos, mientras que cuando se ha llevado a cabo con la CPU, el mismo entrenamiento se ha completado en 9.400 segundos.

El segundo problema encontrado ha sido al tratar de mantener las resoluciones originales para cada imagen de validación. En Tensorflow, como se ha explicado anteriormente, para el conjunto de imágenes se aplican una serie de transformaciones, como son los recortes aleatorios. Estos recortes, se realizan sobre las imágenes del directorio de entrenamiento, por lo que el entrenamiento de las mismas es más rápido al ser las imágenes más pequeñas. Sin embargo, sobre el directorio de validación no se aplican dichas transformaciones, lo que conserva el tamaño original de las imágenes.

En PyTorch, se han intentado reproducir las mismas transformaciones sobre el conjunto de imágenes, encontrando los siguientes obstáculos: a) Al utilizar recortes aleatorios de las imágenes, la red generadora no es capaz de capturar las suficientes características de las imágenes que recibe, y por tanto, no es capaz de generar imágenes de super resolución. Esto se puede deber a que el tamaño de los recortes es demasiado pequeño en relación con las características relevantes de la imagen. Es por ello, que se pierden detalles importantes y el generador no puede aprender a generar imágenes de alta resolución, y b) vuelve a aparecer la falta de memoria al mantener el tamaño original en las imágenes de validación.

5. Resultados

En esta sección, se van a presentar los resultados obtenidos por la red SRGAN implementada tanto en Tensorflow como en PyTorch, donde se van a realizar algunas comparativas entre los resultados de las mismas. Finalmente, se muestran algunas de las imágenes generadas por la red SRGAN implementada en Tensorflow, y la red implementada en PyTorch.

Para empezar, se van a mostrar los resultados que se han obtenido para la red discriminadora. La función de pérdida de esta red está formada por el error cometido prediciendo la imagen real como falsa, y por el error cometido prediciendo la imagen generada como real. Es por ello, que para cada error, se va a presentar un gráfico donde se puede visualizar el error cometido por el generador.

En el primer gráfico se representa el error cometido por el discriminador prediciendo las imágenes reales como falsas en función del tiempo de entrenamiento:

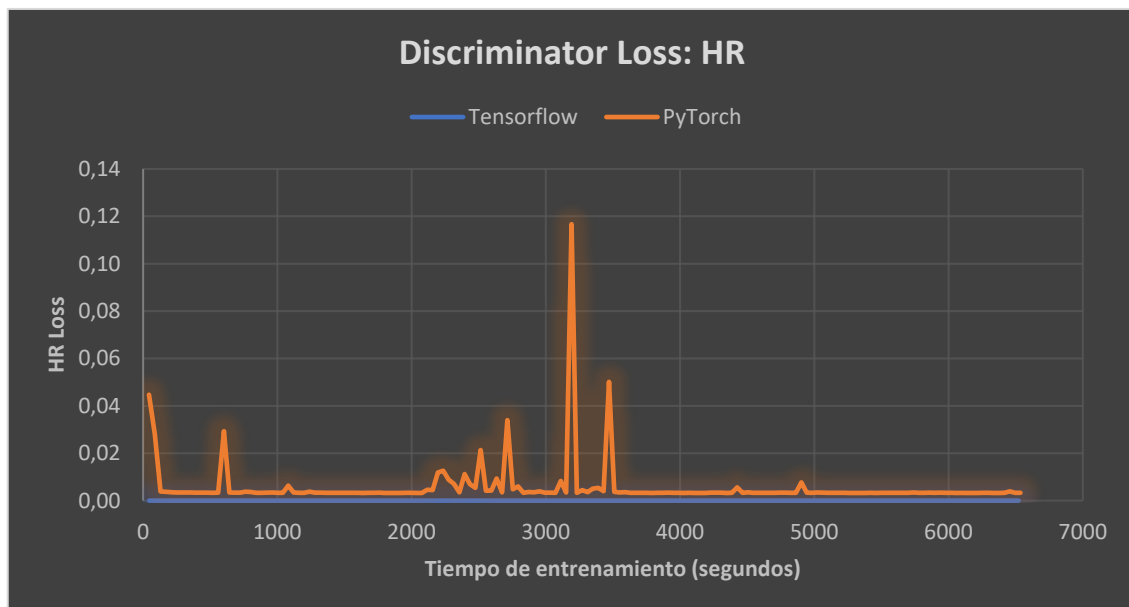


Figura 32. Error cometido por el discriminador con las imagenes reales.

En el caso de Tensorflow, desde el comienzo la red discriminadora es capaz de reconocer cuando una imagen es real, por lo que no tiene error. Sin embargo, en PyTorch, a partir de los 3.000 segundos de entrenamiento, el discriminador llega a confundirse en alguna ocasión con las imágenes reales, y por ello, el error cometido por el mismo es mayor.

En la figura 33, se representa el error cometido por el discriminador prediciendo las imágenes generadas como reales en función del tiempo de entrenamiento:

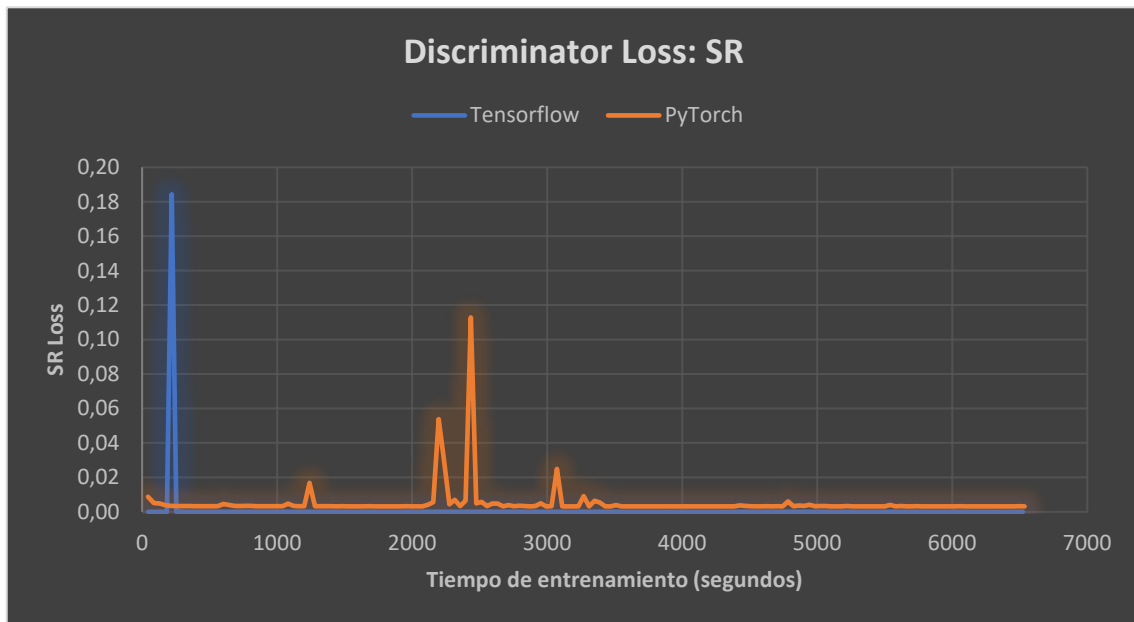


Figura 33. Error cometido por el discriminador con las imagenes generadas.

En esta ocasión, el discriminador en Tensorflow no es perfecto, y ha fallado al comienzo del entrenamiento, donde ha predicho que una imagen generada era real. En el resto de su entrenamiento, el discriminador no ha vuelto a fallar, y mantiene su error en 0. Por otro lado, en PyTorch, el discriminador ha vuelto a confundirse en alguna ocasión prediciendo las imágenes generadas como reales.

A continuación, se van a presentar los resultados obtenidos en el generador. Como ya se ha comentado, la pérdida del generador es en realidad una pérdida perceptual basada en una pérdida de contenido. Esta pérdida se obtiene a través de la red VGG-19. Los valores obtenidos en función del tiempo de entrenamiento son los siguientes:

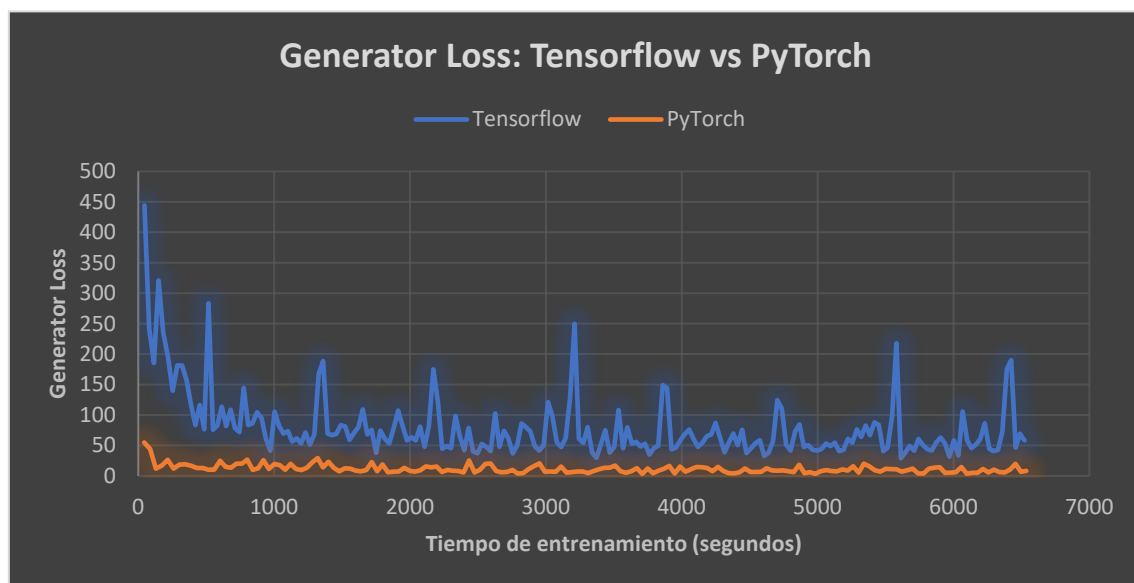


Figura 34. Pérdida del generador basada en la pérdida perceptual.

Aunque en este caso los valores obtenidos en Tensorflow presentan algo de ruido, puede verse con claridad que se encuentra una pérdida de contenido más leve en PyTorch. Esto significa que las características extraídas por la red VGG-19 son más visibles en los resultados producidos por el generador en PyTorch que en Tensorflow.

Para finalizar con las gráficas comparativas, quedaría analizar el error cuadrático medio RMSE en función del tiempo, que realmente es la métrica utilizada para la validación de la red SRGAN.

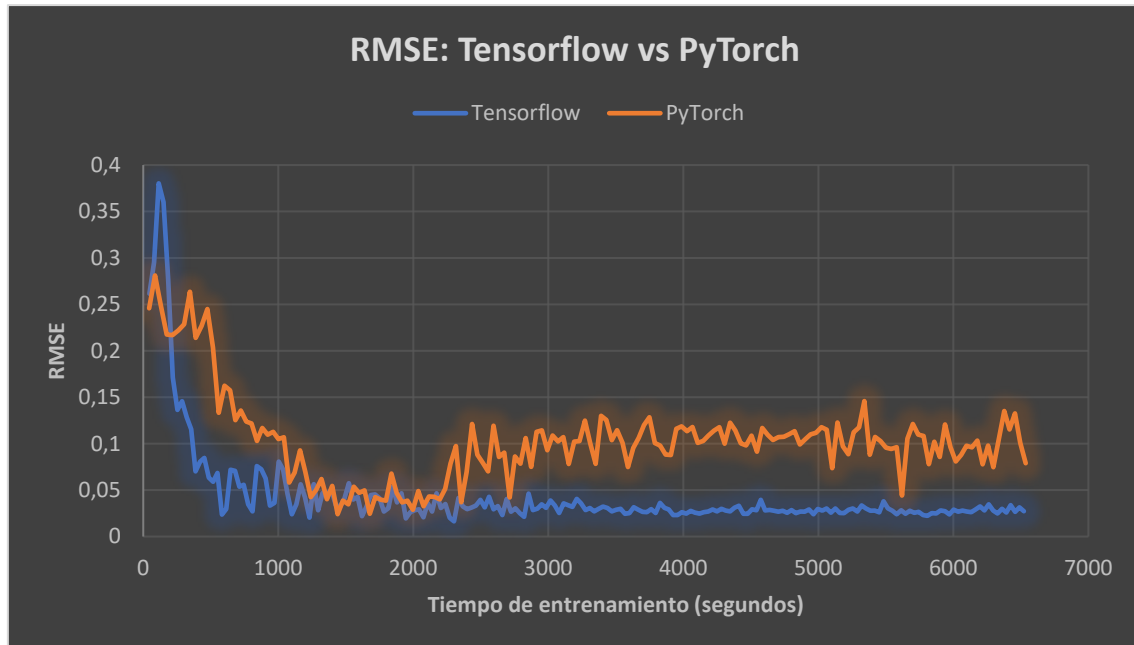


Figura 35. Error cuadrático medio obtenido en las validaciones del entrenamiento.

En este caso, se puede apreciar una leve diferencia entre el error obtenido en Tensorflow y el error obtenido en PyTorch. Al cabo de 7.500 segundos de entrenamiento, Tensorflow alcanza un RMSE de aproximadamente un 2.5%, mientras que PyTorch mantiene un RMSE medio del 9%. Es por ello, que se considera que Tensorflow tiene un mejor rendimiento que PyTorch.

Por último, se van a presentar para cada librería una serie de muestras de las imágenes que ha generado. Se van a presentar imágenes de baja resolución (las imágenes de entrada en el generador), acompañadas de 3 imágenes generadas durante el entrenamiento, y finalmente, se presentará la última imagen producida comparada con la imagen original de alta resolución. En el caso de Tensorflow, se han generado las siguientes:

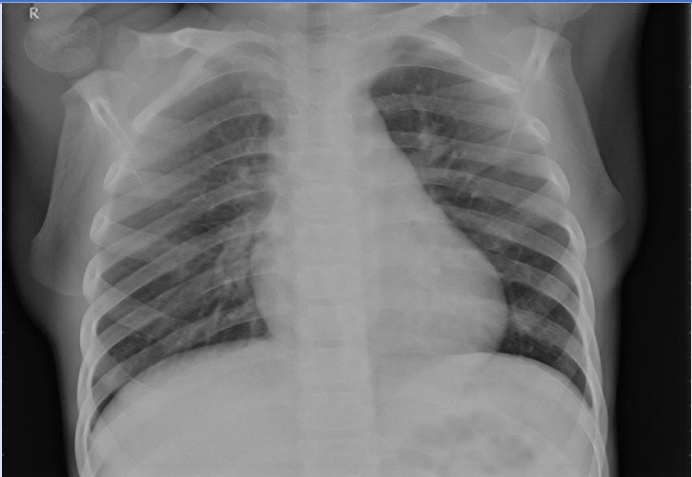
IMAGEN DE ENTRADA DE BAJA RESOLUCIÓN	IMAGEN GENERADA en t=48s
	
IMAGEN GENERADA en t=162s	IMAGEN GENERADA en t=1175s
	
ÚLTIMA IMAGEN GENERADA en t=7.059s	IMAGEN ORIGINAL DE ALTA RESOLUCIÓN
	

Tabla 5. Imágenes obtenidas en Tensorflow.

Para el caso de PyTorch, los resultados (imágenes generadas) que se han obtenido son los que se muestran en la tabla 6:

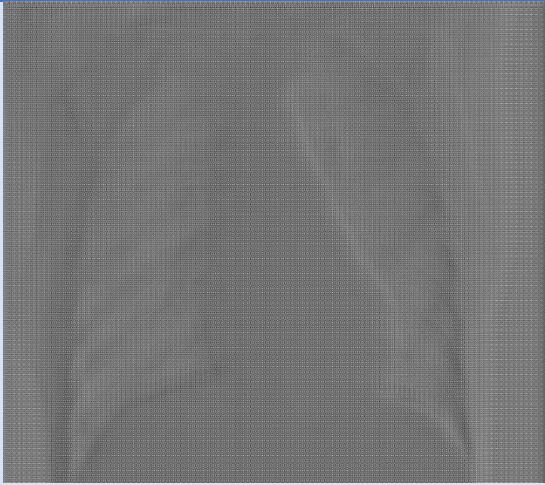
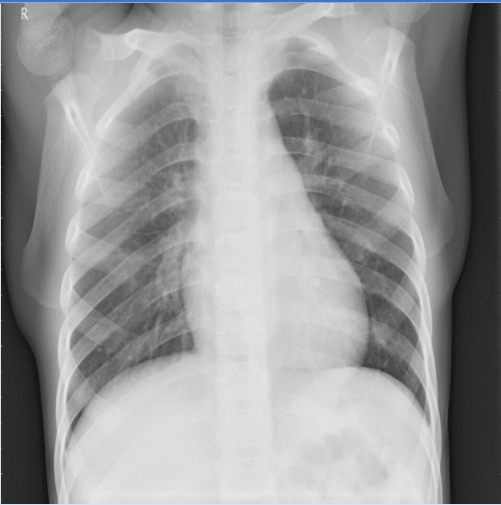
IMAGEN DE ENTRADA DE BAJA RESOLUCIÓN	IMAGEN GENERADA en t=45s
	
IMAGEN GENERADA en t=757s	IMAGEN GENERADA en t=16.462s
	
ÚLTIMA IMAGEN GENERADA en t=25.431s	IMAGEN ORIGINAL DE ALTA RESOLUCIÓN
	

Tabla 6. Imágenes obtenidas en PyTorch.

6. Licencia software y documental

La licencia de software que se va a emplear, es la denominada Berkeley Software Distribution (BSD). Se trata de una licencia de software libre permisiva como puede ser OpenSSL o la MIT License. Existen diferentes tipos de licencias, en el caso de este TFG se ha utilizado la licencia “BSD modificada”, “BSD revisada”, “BSD-3” o “BSD de 3 cláusulas” ^[37].



Figura 36. Logotipo de la licencia BSD.

Por otro lado, este documento ya está amparado por la licencia documental [38] porque forma parte de ZAGUAN, el Repositorio Institucional de Documentos de la Universidad de Zaragoza.

7. Conclusiones y trabajo futuro

En este trabajo se ha abordado el problema de la mejora de imágenes monocromáticas mediante el uso de técnicas de machine learning, en particular con redes generativas adversarias (GAN) y su variante super-resolución (SRGAN). Se han implementado y comparado dos redes SRGAN en los frameworks Tensorflow y PyTorch, utilizando un conjunto de imágenes radiológicas monocromáticas. Se ha evaluado el rendimiento de las redes mediante la métrica RMSE, así como mediante las pérdidas de las redes generadoras y discriminadoras.

Tras ello, se ha determinado que las redes SRGAN son capaces de generar imágenes monocromáticas con una resolución mayor manteniendo la información original.

Por otro lado, se ha podido determinar que las redes SRGAN implementadas en Tensorflow obtienen mejores resultados que las implementadas en PyTorch en términos del RMSE. Esto puede deberse a las diferencias existentes entre las funciones de cada uno de los frameworks.

Además, las redes SRGAN que trabajan directamente con imágenes monocromáticas obtienen mejores resultados que las que trabajan con imágenes policromáticas. Esto se debe a que las imágenes monocromáticas tienen una tercera parte de información que las imágenes policromáticas, y por lo tanto, el entrenamiento es más rápido, y la captura de información es más rápida.

Este TFG resuelve el problema planteado para el que ha sido necesario disponer tanto de conocimientos del área de hardware como de software, adquiridos a lo largo de la titulación del grado en ingeniería informática (GII). La parte más compleja a la que ha habido que enfrentarse ha sido el trabajo con imágenes, en lugar de información textual o numérica, como viene siendo lo habitual con redes neuronales en el campo de la informática.

7.1. Trabajo futuro

Una vez comprobada la importancia de utilizar este tipo de redes para la mejora de imágenes monocromáticas, como trabajo futuro se podrían explorar otras arquitecturas o variantes de redes GAN, como por ejemplo: a) las redes ESRGAN, una mejora de la arquitectura SRGAN para generar texturas aún más realistas o b) WGAN, una variante de GAN que utiliza otras funciones de pérdida para mejorar la estabilidad del entrenamiento y producir muestras aún más realistas. Además, podrían utilizarse otros criterios o métricas para la evaluación del trabajo realizado.

Por último, cabe mencionar la posibilidad de extender el trabajo a otros problemas relacionados con la mejora de imágenes, como por ejemplo el tratamiento del ruido, la posible utilidad de la redundancia de información, etc.

8. Bibliografía ^[39]

[1] Russell, S.J., Norvig, P. and Davis, E. (2020) *Artificial Intelligence: A modern approach*. Harlow, England: Pearson Educación.

[2] José Luis Sarmiento-Ramos. (2020, January 18). *Aplicaciones de las redes neuronales y el Deep learning*. Revista UIS Ingenierías. Retrieved April 15, 2023, from <https://www.redalyc.org/journal/5537/553768213002/html/>

[3] Abascal Sánchez, J. (2019, July). *Mejora estética de imágenes mediante la combinación de redes neuronales convolucionales y algoritmos evolutivos*. Universidad Carlos III de Madrid. Retrieved April 15, 2023, from https://e-archivo.uc3m.es/bitstream/handle/10016/29628/TFG_Juan_Abascal_Sanchez.pdf

[4] Morocho Sande, K. L., & Pulig Cesén, D. X. (2021-03). *Thesis*. Retrieved April 15, 2023, from <http://repositorio.ug.edu.ec/handle/redug/52613>

[5] Ke, R., Zhuang, Y., Pu, Z., & Wang, Y. (2021, August). *Deep Learning Enabled Semantic Communication Systems | IEEE journals*. Retrieved April 15, 2023, from <https://ieeexplore.ieee.org/abstract/document/9398576>

[6] López, J. C. (2021, January 9). *La IA de las tarjetas Gráficas de Nvidia, a prueba: Analizamos el impacto de DLSS 2.0 en el rendimiento y la Calidad de Imagen*. Xataka. Retrieved April 15, 2023, from <https://www.xataka.com/analisis/ia-tarjetas-graficas-nvidia-a-prueba-analizamos-impacto-dlss-2-0-rendimiento-calidad-imagen>

[7] *Introducción Al Aprendizaje Profundo - ML.NET*. Introducción al aprendizaje profundo - ML.NET | Microsoft Learn. <https://learn.microsoft.com/es-es/dotnet/machine-learning/deep-learning-overview>

[8] *Artificial neuron*. Artificial Neuron - GM-RKB. (2023, April 6). Retrieved April 15, 2023, from https://www.gabormelli.com/RKB/Artificial_Neuron

[9] Izaurieta, F., & Saavedra, C. (n.d.). *Redes Neuronales Artificiales*. Retrieved April 15, 2023, from <https://franciscocruz.org/lectures/sistint/rna.pdf>

[10] Martínez Galindo, M. (2019). (rep.). *DeepMyco: Aprendizaje profundo aplicado a la micología*. Teruel: Universidad de Zaragoza.

[11] McMahan, S. (2019, October 23). *Artificial Neural Networks, an introduction*. Data Science Examples. Retrieved April 15, 2023, from <https://www.datascienceexamples.com/artificial-neural-networks-an-introduction-for-data-science-beginners/>

[12] ¿Qué son las redes neuronales? IBM. (n.d.). Retrieved April 15, 2023, from <https://www.ibm.com/es-es/topics/neural-networks>

[13] Logistic function logistic regression sigmoid function statistical classification, Curve Polygon Flyer, angle, white, text PNG. <https://w7.pngwing.com/pngs/976/102/png-transparent-sigmoid-function-equation-angle-sigmoid-curve-angle-text-triangle-thumbnail.png>

[14] de la Torre, J. (n.d.). *REDES GENERATIVAS ADVERSARIAS (GAN): FUNDAMENTOS TEÓRICOS Y APLICACIONES*. Retrieved April 15, 2023, from <https://arxiv.org/ftp/arxiv/papers/2302/2302.09346.pdf>

[15] Google Developers. (2022, September 27). *Descripción General de la Estructura de las Gan*. Google. Retrieved April 15, 2023, from https://developers.google.com/machine-learning/gan/gan_structure

[16] Almagro Blanco, P. (n.d.). Algoritmo de Retropropagación. Retrieved April 15, 2023, from https://www.cs.us.es/~fsancho/ficheros/IAML/2016/Sesion04/capitulo_BP.pdf

[17] Benjaminson, E. (n.d.). *Binary cross-entropy*. Binary Cross-Entropy – Emma Benjaminson – Mechanical Engineering Graduate Student. Retrieved April 15, 2023, from <https://sassafra13.github.io/BiCE/>

[18] Wikimedia Foundation. (2022, October 14). *Teoría de la Información*. Wikipedia. https://es.wikipedia.org/wiki/Teor%C3%ADa_de_la_informaci%C3%B3n

[19] Ledig, C., Theis, L., Huszár, F., Caballero, J., Cunningham, A., Acosta, A., Aitken, A., Tejani, A., Totz, J., Wang, Z., & Shi, W. (2017). *Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network*. IEEE journals. Retrieved April 15, 2023, from <https://arxiv.org/pdf/1609.04802.pdf>

[20] Admin. (2023, May 5). *How to use LeakyReLU as activation function in Keras?*. For Machine Learning. <https://androidkt.com/how-to-use-leakyrelu-as-an-activation-function-in-keras/>

[21] C. Dong, C. C. Loy, K. He, and X. Tang. Image super-resolution using deep convolutional networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 38(2):295–307, 2016

[22] Chen, Z., Pawar, K., Ekanayake, M., Pain, C., Zhong, S., & Egan, G. F. (2022, November 2). Deep learning for image enhancement and Correction in Magnetic Resonance Imaging-State-of-the-art and Challenges - *Journal of Digital Imaging*. SpringerLink. <https://link.springer.com/article/10.1007/s10278-022-00721-9>

[23] Jiménez, Á. de la V. (2022, October 15). *Aplicación de Redes neuronales convolucionales profundas Al Diagnóstico asistido de la enfermedad de Alzheimer*. arXiv.org. <https://arxiv.org/abs/2210.08330v1>

[24] Yadav, A., Singh, G., & Upadhyay, M. (n.d.). Underwater Image Enhancement Using Convolutional neural Network. <https://arxiv-export1.library.cornell.edu/pdf/2109.08916>

[25] Author links open overlay panel Hang Yu a d, a, d, b, c, networks, A. neural, Cui, R., Shen, W., Liu, X., Shen, S., Khan, S. U., Shichijo, S., Zhang, X., Black, M. J., Xie, H., Gu, Y., Zhang, R., Zhang, Y., Charron, O., ... Billones, C. D. (2021, January 26). *Convolutional Neural Networks for Medical Image Analysis: State-of-the-art, comparisons, improvement and perspectives*. Neurocomputing. <https://www.sciencedirect.com/science/article/abs/pii/S0925231221001314>

[26] *Crea Modelos de Aprendizaje Automático de Nivel de Producción Con Tensorflow*. TensorFlow. (n.d.). Retrieved April 15, 2023, from <https://www.tensorflow.org/?hl=es-419>

[27] *PyTorch*. PyTorch. (n.d.). Retrieved April 15, 2023, from <https://pytorch.org/>

[28] *Instalación de TensorFlow*. TensorFlow. (n.d.). Retrieved April 15, 2023, from <https://www.tensorflow.org/install/pip?hl=es-419>

[29] *ISO (International Organization for Standardization)*. Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models. *ISO 25010:2011*. Geneva: ISO, 2011.

[30] *Installing PyTorch*. PyTorch. (n.d.). Retrieved April 15, 2023, from <https://pytorch.org/get-started/locally/>

[31] Mooney, P. (2018, March 24). *Chest X-ray images (pneumonia)*. Kaggle. Retrieved April 25, 2023, from <https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia>

[32] Doñate, P. (n.d.). *Universidad-Zaragoza/Using-ML-techniques-for-image-enhancement*. GitHub. Retrieved April 15, 2023, from <https://github.com/universidad-zaragoza/Using-ML-techniques-for-image-enhancement>

[33] Doñate Navarro, P. (2023, April 27). *ReescalaImagenes.py at using-machine-learning-techniques-for-image-enhancement*. GitHub. Retrieved April 27, 2023, from <https://github.com/universidad-zaragoza/Using-ML-techniques-for-image-enhancement/blob/main/Dataset/ReescalaImagenes.py>

[34] Jlaihong. *Image Super-Resolution using SRResNet and SRGAN*. GitHub. Jupyter Notebook: Train SRResNet and SRGAN.ipynb. https://github.com/jlaihong/image-super-resolution/blob/main/train_SRResNet_and_SRGAN.ipynb

[35] Jlaihong. *Image Super-Resolution*. GitHub. SRResNet.py. <https://github.com/jlaihong/image-super-resolution/blob/main/models/srresnet.py>

[36] Eriklindernoren. (2019, March 30). *PyTorch - GAN*. GitHub. Python File: datasets.py. <https://github.com/eriklindernoren/PyTorch-GAN/blob/master/implementations/srgan/datasets.py>

[37] *The 3-clause BSD License*. Open Source Initiative. (2023, April 7). <https://opensource.org/license/bsd-3-clause/>

[38] *Resumen de Licencia Creative Commons*. Creative Commons - Reconocimiento-NoComercial-SinObraDerivada 3.0 España - CC BY-NC-ND 3.0 ES. (n.d.). <https://creativecommons.org/licenses/by-nc-nd/3.0/es/>

[39] *ISO (International Organization for Standardization)*. Information and documentation – Guidelines for bibliographic references and citations to information resources. *ISO 690:2010*. Geneva: ISO, 2010.

Anexo I. Conceptos básicos de una red neuronal

La arquitectura básica de una neurona artificial es la siguiente:

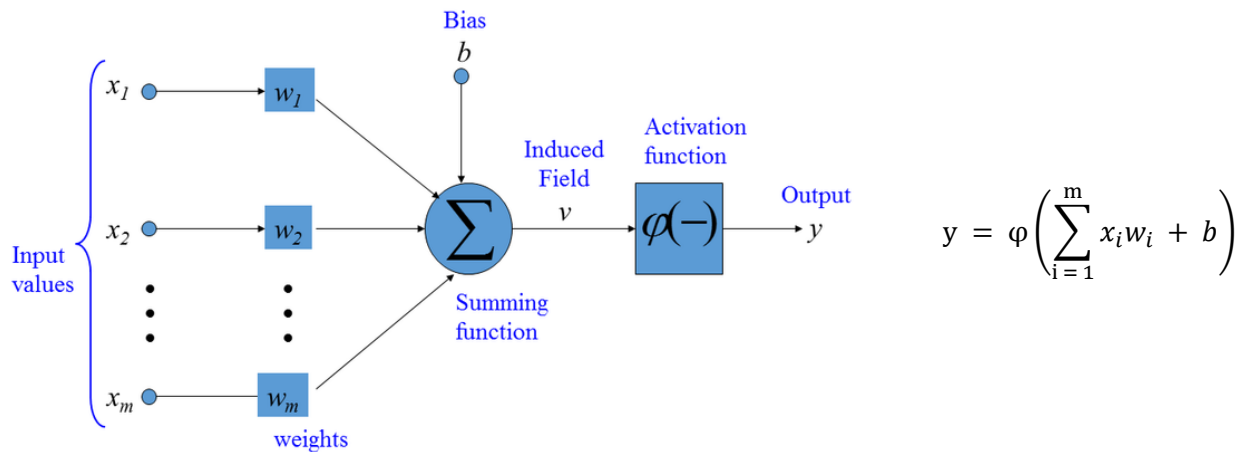


Figura 37. Arquitectura de una neurona artificial ^[8]

Siguiendo la arquitectura representada en la figura 37, ésta recibe diferentes estímulos ^[9] procedentes de otras neuronas $x_1, x_2, \dots, x_m$. Los valores w_i representan los pesos sinápticos entre dos neuronas, de forma que lo que hace cada peso es multiplicarse por su entrada, y finalmente, se suman estos productos junto con otra entrada denominada *bias* y se pasa la suma por una función de activación ϕ .

Una forma sencilla de explicar el funcionamiento de los pesos y la *bias* en un perceptrón es pensar en ellos como una forma de ajustar la entrada para obtener la salida deseada. Los pesos son valores numéricos que se asignan a cada entrada y determinan ^[10] cuánto influirá esa entrada en la salida. Por otro lado, la *bias* es un valor adicional que se agrega a la suma ponderada de las entradas para desplazar la función de activación y permitir que el perceptrón responda mejor a los datos. En el caso de una red neuronal, conforme se entrena la misma, se van ajustando los pesos y las *bias* de las neuronas, de forma que al final del entrenamiento, se tendrá la combinación de valores que más se ajustan para resolver el problema para el que se ha entrenado.

Se pueden intentar solucionar problemas aritmético-lógicos básicos con una neurona, como por ejemplo, aislar los valores de las puertas *AND*, *OR* y *XOR*. A continuación, se van a resolver estos problemas para poder entender la finalidad de una neurona.

Se va a suponer que la función de activación es una función escalonada.

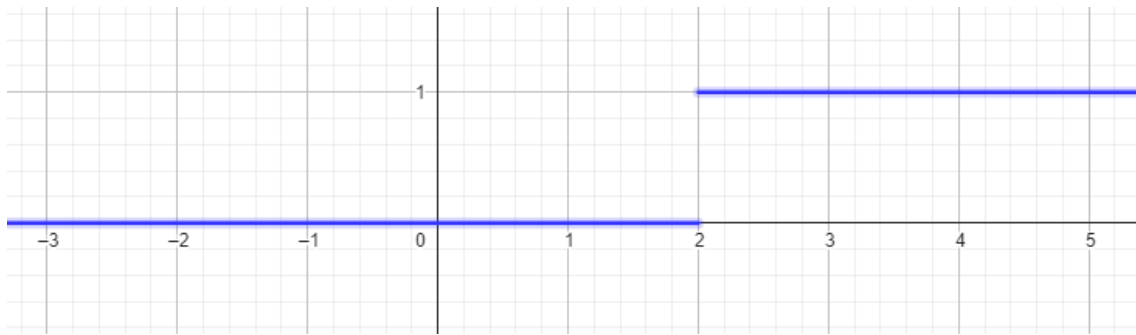


Figura 38. Función de activación Escalón.

Esta función devolverá 1 si supera un umbral determinado y 0 en caso contrario. En el ejemplo siguiente se supone que ese umbral es 2, de forma que, si el resultado es mayor o igual que 2, devolverá 1.

$$f(y^{in}) = \begin{cases} 0, & \text{if } y^{in} < 2 \\ 1, & \text{if } y^{in} \geq 2 \end{cases}$$

Ahora se va a suponer que se tiene una neurona para solucionar la puerta AND. La tabla de verdad que representa la puerta lógica (los valores con los que trabaja) y la neurona son las siguientes:

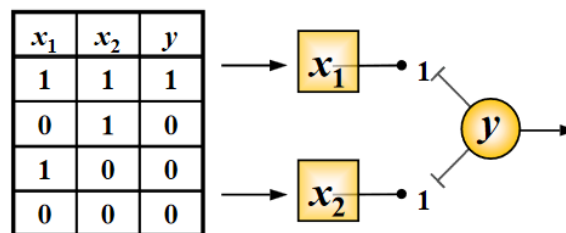


Figura 39. Puerta lógica AND ^[9]

De esta forma, si se verifica la salida que tendría la neurona con los datos de entrada de la tabla de verdad, por ejemplo de la primera fila ($x_1 = 1$, $x_2 = 1$), se tendrá lo siguiente:

$$y^{in} = 1 \cdot 1 + 1 \cdot 1 = 2$$

Como $y^{in} \geq 2$, $f(y^{in}) = 1$, se puede verificar que se cumple.

Ahora, se va a solucionar la puerta OR mediante otra neurona, siendo la tabla de verdad (los valores con los que trabaja) y la neurona los siguientes:

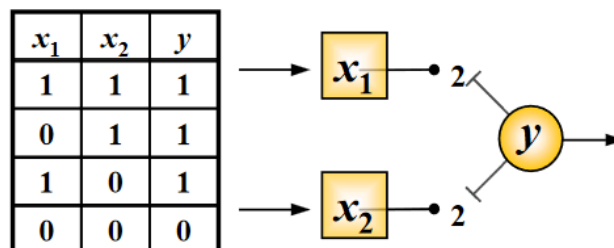


Figura 40. Puerta lógica OR ^[9]

De esta forma, se vuelve a verificar la salida que tendría la neurona con los datos de entrada de la segunda fila de la tabla de verdad ($x_1 = 0, x_2 = 1$), se tendrá lo siguiente:

$$y^{in} = 0 \cdot 2 + 1 \cdot 2 = 2$$

Como $y^{in} \geq 2$, $f(y^{in}) = 1$, se puede verificar que se cumple.

Sin embargo, si se intenta solucionar la puerta *XOR* mediante una única neurona como se ha hecho en los casos anteriores, se puede ver que no se encuentra ninguna combinación de valores para los pesos que satisfaga la tabla de verdad de la puerta lógica. Esto es debido a que la *XOR* no es un problema linealmente separable, es decir, no se puede encontrar una línea recta que separe los valores positivos de los negativos en un gráfico. Si se representa la puerta *AND* sobre el plano sería el siguiente, donde se puede ver que existen rectas que la satisfacen:

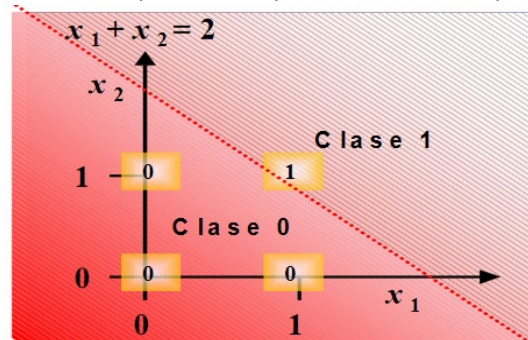


Figura 41. Puerta AND sobre el plano ^[9]

En cuanto a la puerta *XOR* no existe ninguna recta que deje a un lado las entradas que producen 0, y al otro lado las que producen 1. La puerta *XOR* sobre el plano sería la siguiente:

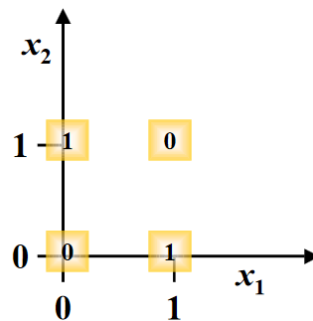


Figura 42. Puerta XOR ^[9]

De esta característica, surgió la idea de conectar dos neuronas, de forma que la primera de ellas, separa linealmente la puerta *OR*, y la segunda neurona la puerta *AND*. Así, se conseguía solucionar el problema. Esto provocó un cambio de pensamiento, en el que un conjunto de neuronas conectadas (red neuronal) podría resolver problemas más complejos de los previstos hasta el momento.

Anexo II. Retropropagación

El concepto de retropropagación o *backpropagation* ^[16], se basa en la idea de “volver hacia atrás” en el aprendizaje. ⁴Es decir, en lugar de actualizar todos los pesos y *bias* en cada iteración, el algoritmo vuelve hacia atrás intentando entender qué neuronas han tenido más peso a la hora de decidir la salida. De esta forma, el algoritmo sólo actualiza los pesos y *bias* que han tenido importancia en la salida obtenida: ⁵

1. Para cada capa l , calcula z^l (la entrada ponderada a las neuronas de la capa l), que viene dada por:

$$z^l = w^l a^{l-1} + b^l$$

w^l son los pesos de la capa, a^{l-1} es la activación de la capa anterior y b^l es la *bias* de la capa.

2. Cada vez que calcula z^l , actualiza la activación de esa capa calculando la función sigmoide:

$$a^l = \sigma(z^l)$$

3. Después de calcular a^l para cada capa, calcula el error delta de la última capa.

Dada una función de coste C que evalúa la similitud entre la salida obtenida de la red a^L y la salida esperada y , el objetivo del algoritmo es calcular las derivadas parciales $\frac{\partial C}{\partial w}$ y $\frac{\partial C}{\partial b}$ las cuales permiten modificar los pesos y *bias* de cada neurona para reducir el coste C . Se define δ^l como el error delta de la capa l , es decir, cuanto varía la función de coste con respecto a las entradas de las neuronas de dicha capa. El error delta para la última capa será el siguiente:

$$\delta^L = \nabla_a C \odot \sigma'(z^L)$$

El producto Hadamard ($\odot$) toma dos matrices de iguales dimensiones y produce una nueva matriz donde cada elemento ij es el producto de los elementos ij en las dos matrices originales.

Además, $\nabla_a C$ es el vector compuesto por las derivadas parciales de C con respecto a las diferentes activaciones de las neuronas en la capa de salida. σ' es la derivada de la función sigmoide ($\frac{\partial a^L}{\partial z^L}$).

4. Por último, vuelve hacia atrás con el error, calculando los errores delta de las demás capas. Se debe de hacer una distinción entre el error delta de la capa de salida L y del resto de capas $L-1, L-2, \dots, 1$. El error delta para el resto de capas será el siguiente:

$$\delta^l = ((w^{l+1})^T \delta^{l+1}) \odot \sigma'(z^l)$$

Donde $(w^{l+1})^T$ representa la matriz transpuesta de la matriz de pesos de la capa $l+1$, necesaria para propagar hacia atrás los errores.

Una vez haya llegado a la primera capa, el algoritmo habrá terminado, y habrá que determinar el gradiente de la función de coste C . La ecuación que indica cómo cambia C con respecto a cualquier *bias* en la red:

$$\frac{\partial C}{\partial b_j^l} = \delta_j^l$$

y la ecuación que indica cómo cambia el coste C con respecto a cualquier peso en la red, sería la siguiente:

$$\frac{\partial C}{\partial w_{jk}^l} = a_k^{l-1} \delta_j^l$$

Así es como calcula el algoritmo de *backpropagation* el gradiente de la función de pérdida para un ejemplo de entrenamiento.

⁴ Aunque el concepto del *backpropagation* no es nuevo, su funcionalidad práctica no se demostró hasta 1986. Gracias a ello alcanzó mucha popularidad.

⁵ Demostración funcionamiento *backpropagation*: <https://www.linkedin.com/learning/fundamentos-de-inteligencia-artificial-redes-neuronales/ajusta-la-red-neuronal>